Comprehensive Computer Vision and Deep Learning Report

Group 7

Aditya Kumar (240059)

Ahmad Akhtar (240074)

Divy Kumar Pandey (240368)

Rudraksh Kumawat (240891)

Kshitij Jain (240570)

Kunal Garhewal (240581)

Manthan Gupta (240629)

Parth Rathi (240736)

Saket Kumar (240908)

Vinay Jangid (241165)

Contents

1 Introduction to Computer Vision

In Computer Vision, images are treated as numerical data rather than visual pictures. This numerical formulation enables the application of mathematical techniques for image enhancement, filtering, segmentation, recognition, and compression. At the core of modern vision systems lie two complementary perspectives: the spatial domain, which represents images as pixel intensity values, and the frequency domain, which represents images as combinations of sinusoidal components of varying frequencies. Understanding these representations is fundamental for analyzing image structure, suppressing noise, enhancing features, and reducing redundancy in deep neural networks.

2 Images as Mathematical Objects

2.1 Digital Image Representation

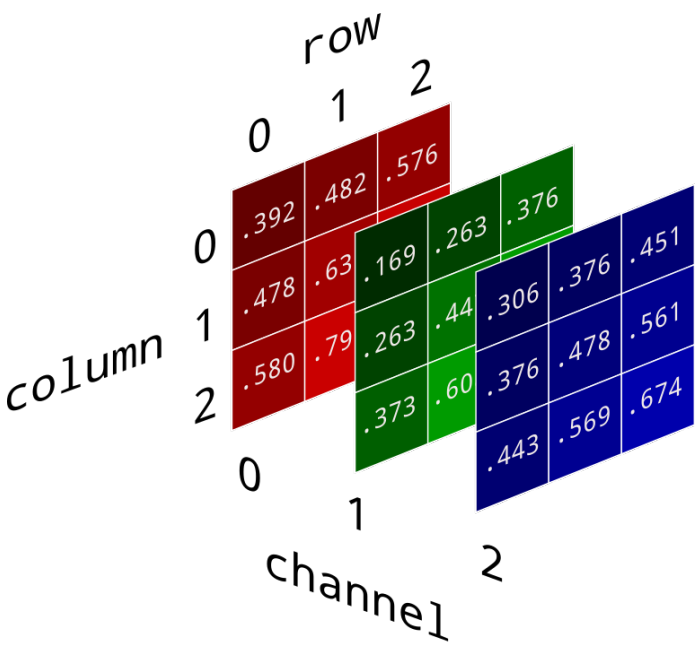
A digital color image can be represented as:

$$\text{Image} \in \mathbb{R}^{H \times W \times 3}$$

where H and W denote the height and width of the image, and the third dimension corresponds to the three color channels. Each pixel is a triplet (R, G, B) with:

$$0 \leq R, G, B \leq 255$$

in typical 8-bit image formats. For numerical processing, these values are often converted to floating-point representations and normalized to the range $[0, 1]$ to improve numerical stability and computational accuracy.



2.2 Images as Matrices

A grayscale image can be modeled as a two-dimensional matrix:

$$I(x, y) \in \mathbb{R}^{H \times W}$$

where each element represents the intensity at spatial location (x, y). A color image is equivalently represented as three such matrices stacked along the channel dimension:

$$I(x, y) = \{R(x, y), G(x, y), B(x, y)\}$$

This matrix formulation enables the use of linear algebra, convolution, filtering, and spectral analysis. Operations such as smoothing, sharpening, edge detection, and frequency transformation are all expressed as matrix operations or matrix-based transforms, making images natural objects of mathematical analysis.

3 RGB, BGR, and Grayscale Images

3.1 RGB Representation

The RGB color model represents each pixel as a combination of red, green, and blue intensities. By varying the magnitude of these three components, a wide range of colors can be

synthesized. RGB is widely used in digital imaging and display systems because it aligns with the physical properties of light emission in electronic screens.

3.2 BGR Representation

In some computer vision libraries, such as OpenCV, images are stored in BGR format, where the channel order is Blue, Green, and Red. Although numerically different from RGB ordering, the underlying image information remains unchanged; only the channel interpretation differs. Care must be taken when performing color-based processing to ensure correct channel interpretation.

3.3 Grayscale Representation

A grayscale image contains a single channel that represents luminance or intensity. By discarding color information, grayscale images preserve structural content such as edges, contours, and textures while significantly reducing memory and computational requirements. As a result, grayscale representations are widely used in edge detection, feature extraction, segmentation, and frequency-domain analysis.

4 Grayscale Conversion

4.1 Why Grayscale?

Converting a color image to grayscale simplifies processing by reducing data dimensionality while retaining essential spatial information. Since many vision tasks depend primarily on intensity variations rather than chromatic differences, grayscale images are often sufficient for analyzing shapes, edges, and textures.

4.2 Mathematical Formulation

Grayscale conversion is defined as:

$$I_{\text{gray}} = 0.299R + 0.587G + 0.114B$$

This weighted sum reflects the sensitivity of the human visual system, which is most responsive to green, followed by red, and least sensitive to blue. The resulting grayscale image provides a perceptually meaningful measure of brightness.

Figure 1: Original RGB Image and Corresponding Grayscale Image

5 Spatial Domain vs Frequency Domain

5.1 Spatial Domain

In the spatial domain, image processing operations act directly on pixel values. Techniques such as convolution with smoothing kernels, sharpening masks, thresholding, and morphological operations modify local neighborhoods of pixels. While spatial methods are intuitive and effective for many tasks, they operate locally and may not explicitly reveal global repetitive patterns or frequency content.

5.2 Frequency Domain

In the frequency domain, an image is represented as a sum of sinusoidal components with different frequencies and orientations. Low frequencies correspond to smooth regions and gradual intensity variations, whereas high frequencies correspond to edges, fine textures, and noise. Frequency-domain analysis enables selective enhancement or suppression of specific components, making it a powerful tool for denoising, compression, and feature extraction.

6 Discrete Fourier Transform (DFT)

The Discrete Fourier Transform converts a spatial-domain image into its frequency representation. For a 2D image $I(x, y)$ of size $M \times N$, the DFT is defined as:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} I(x, y) e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

Each coefficient $F(u, v)$ represents the contribution of a sinusoidal pattern with horizontal frequency u and vertical frequency v.

6.1 Interpretation of DFT Components

The DFT decomposes an image into a set of orthogonal basis functions corresponding to different spatial frequencies. Low-frequency components encode coarse image structure such as illumination and large homogeneous regions, while high-frequency components capture sharp transitions, edges, and noise. This decomposition provides a global characterization of image content.

6.2 Time Complexity of DFT

Direct computation of the DFT requires evaluating MN frequency coefficients, each involving a summation over MN spatial points:

$$O(MN \cdot MN) = O(N^2)$$

for an $N \times N$ image. This quadratic complexity makes naive DFT computation impractical for large images.

7 Fast Fourier Transform (FFT)

The Fast Fourier Transform is an algorithmic optimization that exploits symmetry and periodicity in the Fourier basis functions to reduce computational complexity. By recursively decomposing the DFT into smaller subproblems, FFT reduces the complexity from $O(N^2)$ to:

$$O(N \log N)$$

This dramatic improvement enables real-time frequency-domain processing in applications such as filtering, compression, and deep learning pipelines.

7.1 Inverse FFT

After frequency-domain manipulation, the original image is reconstructed using the Inverse Fourier Transform:

$$I(x, y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

The inverse FFT converts the modified frequency representation back into the spatial domain, allowing the effects of filtering or enhancement to be visualized.

8 Magnitude and Phase

8.1 Magnitude Spectrum

The magnitude spectrum is computed as:

$$|F(u,v)| = \sqrt{\mathrm{Re}(F(u,v))^2 + \mathrm{Im}(F(u,v))^2}$$

To improve visualization, logarithmic scaling is applied:

$$M(u,v) = \log(|F(u,v)| + 1)$$

The magnitude spectrum represents the distribution of energy across frequencies. Bright regions correspond to dominant frequency components. Typically, low frequencies appear near the center of the spectrum after shifting, indicating overall illumination and smooth variations, while high frequencies appear toward the edges, corresponding to fine textures and abrupt intensity changes.

8.2 Phase Spectrum

The phase spectrum is defined as:

$$\Phi(u,v) = \angle F(u,v) = \tan^{-1}\left(\frac{\mathrm{Im}(F(u,v))}{\mathrm{Re}(F(u,v))}\right)$$

Phase encodes the spatial alignment of frequency components. While magnitude indicates *how much* of a frequency is present, phase determines *where* that frequency appears in the image.

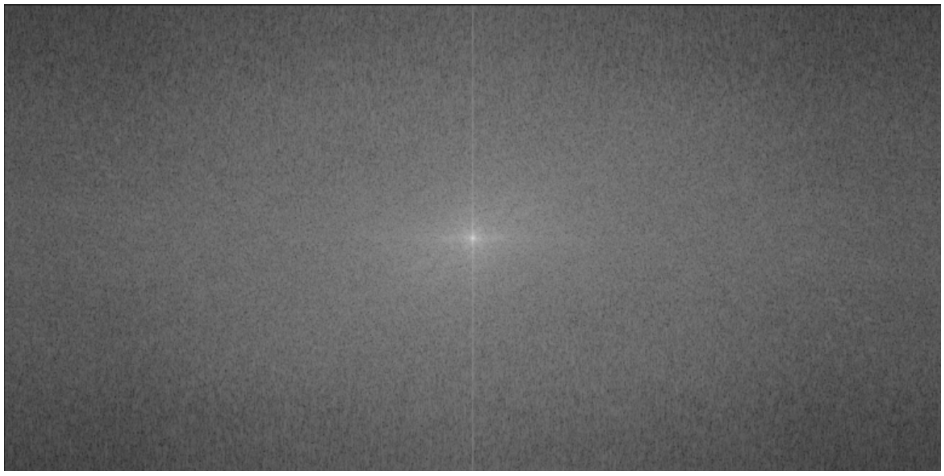


Figure 2: Magnitude Spectrum

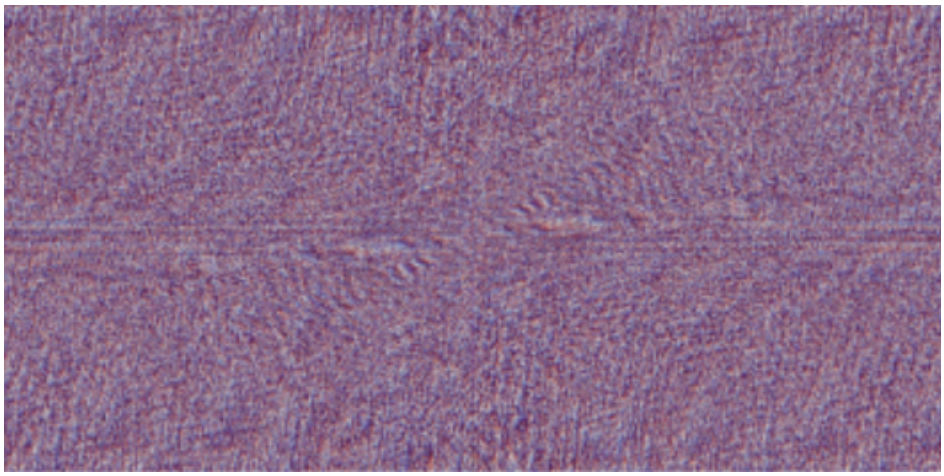


Figure 3: Phase Spectrum

8.3 Procedure for Obtaining Magnitude and Phase Plots

To compute and visualize magnitude and phase:

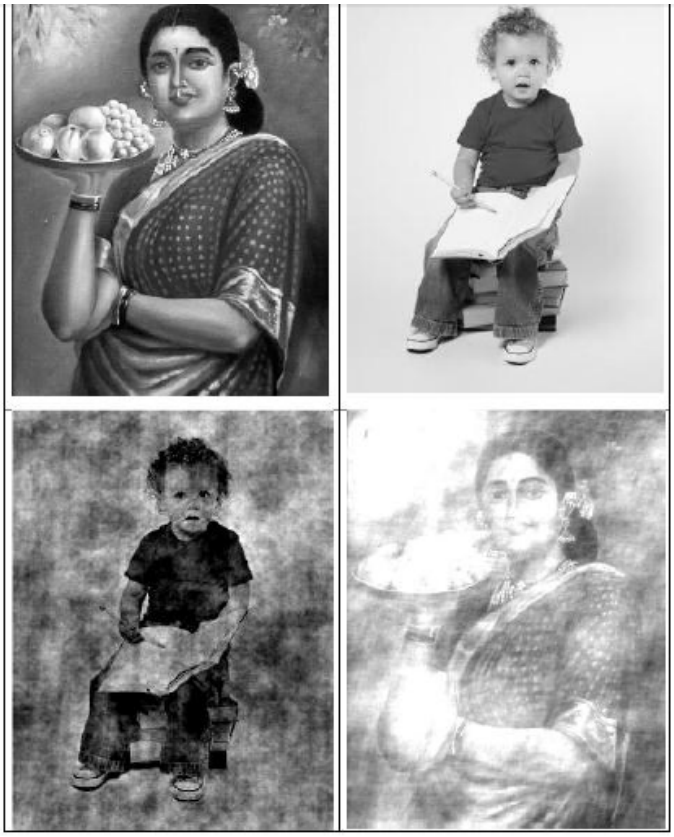
1. Convert the image to grayscale and floating-point representation.
2. Apply FFT to obtain $F(u, v)$.
3. Shift the zero-frequency component to the center of the spectrum.
4. Compute the magnitude spectrum $|F(u, v)|$ and apply logarithmic scaling.
5. Compute the phase spectrum using $\tan^{-1}(\text{Im}/\text{Re})$.
6. Display both spectra using appropriate normalization for visualization.

8.4 Why Phase Preserves Structure

Phase determines how sinusoidal components align spatially to reconstruct image features. If the phase information is removed or randomized while keeping the magnitude intact, the reconstructed image loses recognizable shapes and spatial organization. Conversely, if phase is preserved and magnitude is modified, the resulting image still retains major structural elements such as edges, contours, and object boundaries. This demonstrates that phase encodes the geometric and positional layout of the image.

8.5 Why Magnitude Preserves Texture

Magnitude specifies the strength of each frequency component. High-frequency magnitudes correspond to fine textures, edges, and noise, while low-frequency magnitudes represent smooth intensity variations and coarse shading. Therefore, magnitude primarily governs texture, contrast, and energy distribution in the image, rather than the precise spatial arrangement of structures.



a	b
c	d

(a) test image1 (b) test image2 (c) Image Reconstruction from Magnitude of Image1 and Phase of Image2 (d) Image Reconstruction from Magnitude of Image2 and Phase of Image1

Figure 4: Magnitude and Phase Components

9 Image Filtering in Frequency Domain

9.1 Low-Pass Filter

A Low-Pass Filter (LPF) allows low-frequency components to pass while attenuating high-frequency components:

$$H(u,v) = \begin{cases} 1, & D(u,v) \leq D_0 \\ 0, & D(u,v) > D_0 \end{cases}$$

where $D(u,v)$ is the distance from the frequency origin and D_0 is the cutoff frequency. LPFs smooth images, reduce noise, suppress fine details, and are widely used for denoising and background extraction.

9.2 High-Pass Filter

A High-Pass Filter (HPF) retains high-frequency components while suppressing low frequencies:

$$H(u,v) = \begin{cases} 0, & D(u,v) \leq D_0 \\ 1, & D(u,v) > D_0 \end{cases}$$

HPFs emphasize edges, highlight object boundaries, and enhance fine details. They are particularly useful for feature enhancement, sharpening, and edge detection.

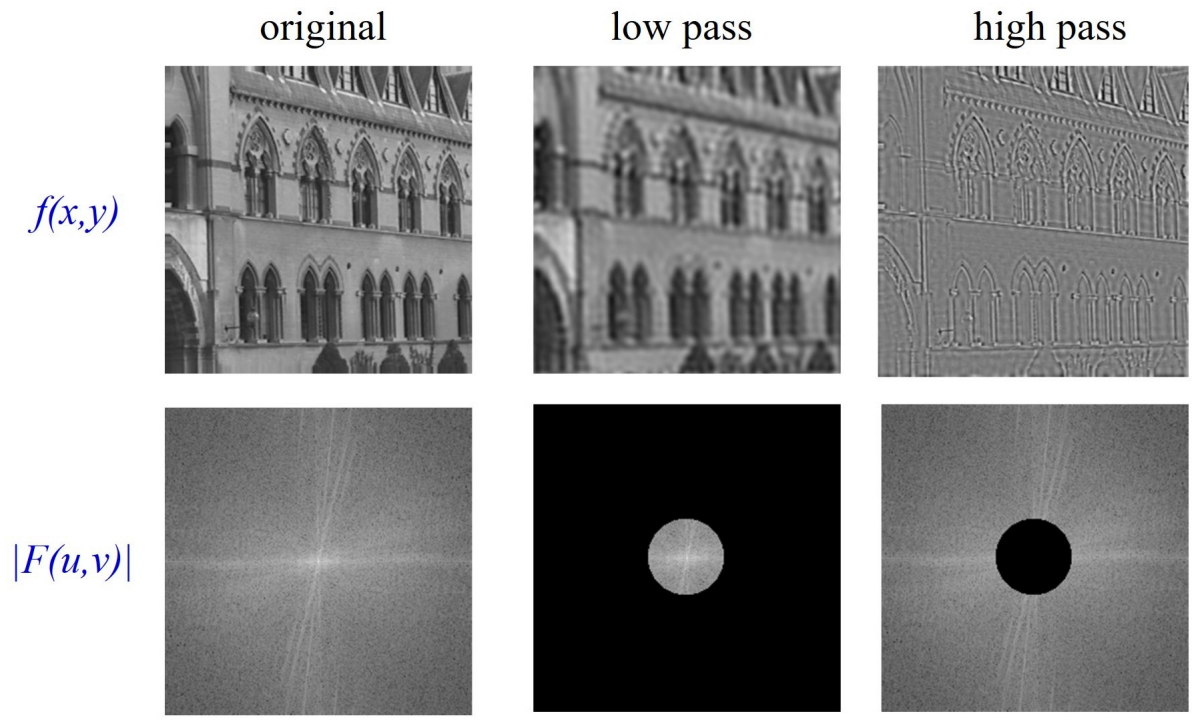


Figure 5: Effect of Low-Pass and High-Pass Filtering

10 Applying Frequency Masks

Frequency-domain filtering involves the following steps:

1. Convert the image to floating-point format.
2. Apply FFT to transform the image to the frequency domain.
3. Shift the zero-frequency component to the center.
4. Multiply the spectrum by a frequency mask (LPF or HPF).

5. Apply inverse shifting.

6. Apply inverse FFT to return to the spatial domain.

7. Take the absolute value of the reconstructed image.

This procedure enables selective manipulation of image components based on their frequency content.

11 Histograms

A histogram is one of the most fundamental tools in image processing. It provides a global summary of how pixel intensities are distributed in an image. Rather than focusing on the spatial arrangement of pixels, a histogram simply counts how many pixels fall into each intensity level.

For a digital image, the horizontal axis represents possible intensity values (typically from 0 to 255 for 8-bit images), while the vertical axis represents the number of pixels that possess each intensity. In essence, a histogram is a compact statistical description of the image's brightness distribution.

From the shape of a histogram, we can infer important visual properties of an image: if most values are clustered toward the left, the image appears dark; if they are clustered toward the right, the image appears bright. A narrow histogram indicates low contrast, while a wide histogram indicates high contrast. Thus, histograms are widely used in exposure analysis, contrast enhancement, thresholding, and many preprocessing pipelines.

Two commonly used types of histograms are described below.

1. **Grayscale Histogram:** For a grayscale image, the histogram has a single channel. The X-axis corresponds to intensity values from 0 (pure black) to 255 (pure white), and the Y-axis gives the number of pixels at each intensity. This histogram represents the overall brightness distribution of the image and is frequently used for operations such as histogram equalization and adaptive thresholding.

2. **RGB Histogram:** For a color image, separate histograms are computed for the Red, Green, and Blue channels. Each channel histogram describes the distribution of intensities for that particular color component. By comparing these three histograms, one can identify color dominance, imbalance, or cast in the image. For example, an image with consistently higher red intensities will appear warm, whereas one with higher blue values will appear cool.

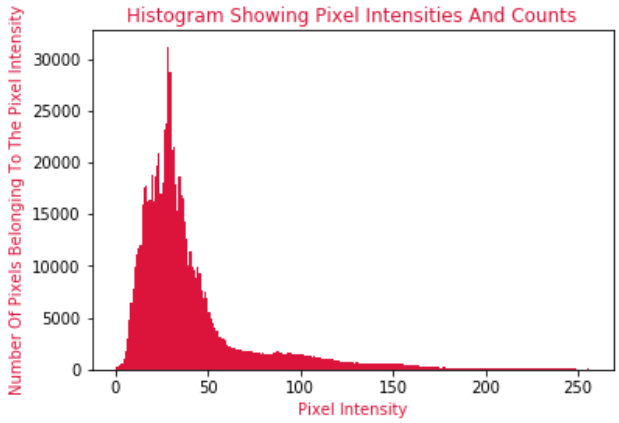


Figure 6: Grayscale Histogram

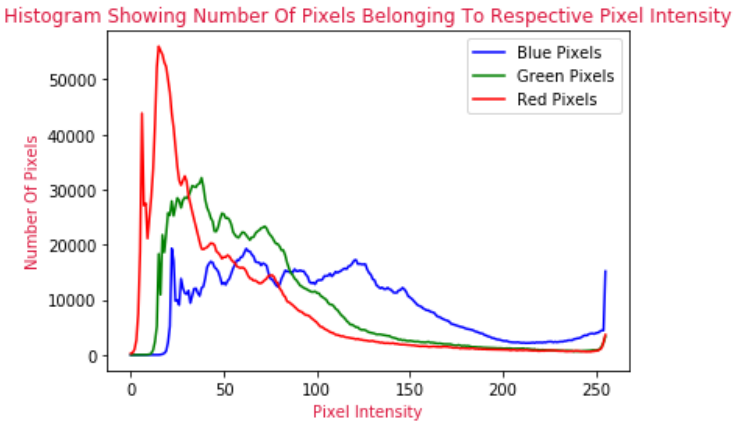


Figure 7: RGB Histogram

12 Colour Spaces

A color space is a mathematical representation of colors that allows color values to be described as tuples of numbers. Different color spaces are designed for different purposes: some are hardware-oriented (such as RGB), while others are perceptually motivated (such as HSV and HSL).

1. **RGB (Red, Green, Blue):** This is the most common color space in digital imaging and display systems. An image is represented as three matrices corresponding to the red, green, and blue components. Each pixel is described by a triplet (R, G, B). Although RGB is efficient for hardware and storage, it does not align well with how humans perceive color differences.

2. **HSV (Hue, Saturation, Value):** HSV represents color in a way that is closer to human intuition. *Hue* represents the type of color (red, green, blue, etc.) and is measured as an angle Ψ on the color wheel. *Saturation* describes how vivid or pure the color is, while *Value* corresponds to brightness.

3. **HSL (Hue, Saturation, Lightness):** HSL is similar to HSV but replaces Value with *Lightness*, which measures the intensity of a color relative to both black and white. Lightness takes its maximum at white, minimum at black, and true color at the midpoint.

Hue represents the angular displacement from the red reference on the color wheel. Saturation quantifies color purity: a saturation of zero corresponds to grayscale, while higher values produce more vivid colors. The distinction between HSV and HSL lies in how brightness is defined. In HSV, brightness is determined by the strongest RGB component:

$$V = \max(R, G, B),$$

whereas in HSL, lightness is computed as the midpoint between the maximum and minimum intensity values:

$$L = \frac{\max(R, G, B) + \min(R, G, B)}{2}.$$

As a result, saturation in HSL is strongest at $L = 0.5$ and weak near the extremes, while in HSV it scales with brightness.

13 RGB to HSV Conversion

The conversion from RGB to HSV is frequently required in computer vision tasks, especially when color segmentation, illumination normalization, or perceptual color manipulation is needed. Below is the standard algorithmic procedure.

13.1 Step 0: Normalization

Given RGB values in the range $[0, 255]$, they are first normalized:

$$R' = \frac{R}{255}, \quad G' = \frac{G}{255}, \quad B' = \frac{B}{255}.$$

13.2 Step 1: Compute Extrema and Delta

$$C_{\max} = \max(R', G', B'), \quad C_{\min} = \min(R', G', B'), \quad \Delta = C_{\max} - C_{\min}.$$

13.3 Step 2: Value (V)

The Value channel represents brightness:

$$V = C_{\max}.$$

13.4 Step 3: Saturation (S)

$$S = \begin{cases} 0, & C_{\max} = 0, \\ \frac{\Delta}{C_{\max}}, & C_{\max} \neq 0. \end{cases}$$

13.5 Step 4: Hue (H)

$$H = \begin{cases} 0, & \Delta = 0, \\ 60^\circ \times \left(\frac{G'-B'}{\Delta} \mod 6 \right), & C_{\max} = R', \\ 60^\circ \times \left(\frac{B'-R'}{\Delta} + 2 \right), & C_{\max} = G', \\ 60^\circ \times \left(\frac{R'-G'}{\Delta} + 4 \right), & C_{\max} = B'. \end{cases}$$

If H is negative, 360° is added to ensure $H \in [0^\circ, 360^\circ)$.

13.6 Final HSV Ranges

Channel	Range
H	$0^\circ - 360^\circ$
S	$0 - 1$
V	$0 - 1$

(Some libraries scale these values, e.g., OpenCV uses $H \in [0, 179]$ and $S, V \in [0, 255]$.)

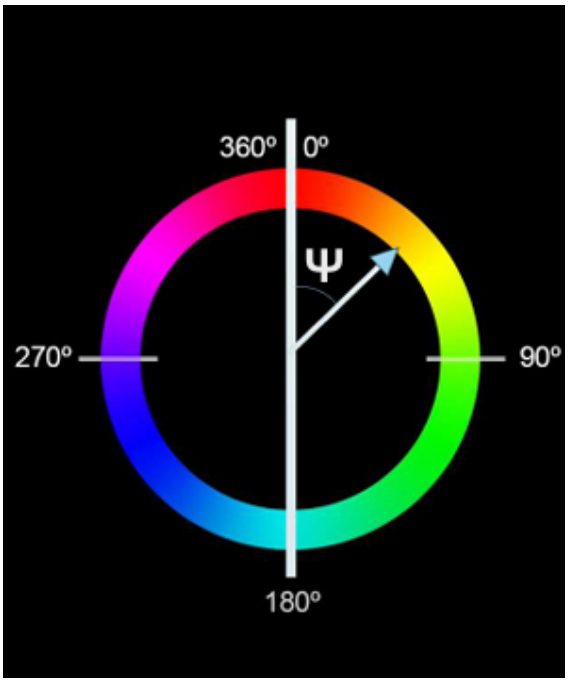


Figure 8: HSV Wheel

14 Image Properties

14.1 Brightness

Brightness refers to the overall intensity of pixels in an image. In HSV space, it corresponds to the Value channel. Computationally, brightness adjustment is performed by adding a constant β to every pixel:

$$I'_{ij} = \text{clip}(I_{ij} + \beta, 0, 255).$$

Increasing brightness shifts the histogram to the right but does not increase contrast. Excessive brightening may cause highlights to clip, while darkening may suppress shadow details.

14.2 Contrast

Contrast measures the difference between the dark and bright regions of an image. Increasing contrast stretches the histogram, enhancing visual depth, while decreasing contrast compresses the histogram:

$$I'_{ij} = \text{clip}(\alpha(I_{ij} - \text{Midpoint}) + \text{Midpoint}, 0, 255).$$

Here, $\alpha > 1$ increases contrast, and $\alpha < 1$ reduces it.

14.3 Saturation

Saturation defines the vividness of color. A saturation of zero produces grayscale, while higher saturation yields more intense colors. Over-saturation can cause channel clipping and loss of detail.

14.4 Hue

Hue corresponds to an angular rotation on the color wheel. Modifying hue changes color identity while preserving brightness, saturation, and texture. For example, rotating hue by 120° maps RGB colors to BRG.

14.5 Gamma Correction

Gamma correction accounts for the nonlinear response of display devices and human vision. Since humans are more sensitive to darker tones, pixel intensities are often stored in gamma-compressed form. The correction is given by:

$$I_{\text{out}} = I_{\text{in}}^{\gamma}.$$

For legacy CRT displays, $\gamma \approx 2.2$.

14.6 White Balance

White balance compensates for color casts introduced by lighting conditions, ensuring that objects that should appear white are rendered neutrally. Common approaches include:

- **Gray World Assumption:** assumes the average color of the image is gray.
- **White Patch Method:** assumes the brightest pixel should be white.
- **Learning-Based Methods:** use trained models to estimate the scene illuminant.

14.7 Vibrance

Unlike saturation, which boosts all colors equally, vibrance selectively enhances low-saturation pixels while preserving already vivid colors and protecting skin tones. This results in more natural-looking enhancement.

15 Gradients

A gradient measures how rapidly pixel intensity changes and in which direction. In one dimension, it approximates the difference between neighboring pixels. The strength of an edge is given by the gradient magnitude:

$$|\nabla I| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

and the orientation of the edge is:

$$\theta = \tan^{-1}\left(\frac{\partial I/\partial y}{\partial I/\partial x}\right)$$

Edges correspond to locations where the gradient magnitude is large.
Several discrete operators approximate image gradients:

- **Roberts Cross:** small, fast, but noise sensitive.
- **Prewitt:** includes averaging for better noise robustness.
- **Sobel:** uses weighted smoothing, making it the most commonly used operator.

16 Convolution

Convolution is the core operation in spatial filtering. It replaces each pixel by a weighted sum of its neighbors using a kernel:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j)\, K(i, j).$$

By selecting different kernels, different effects are obtained.

16.1 Average Blur

Applies equal weights to all neighbors:

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

This reduces noise but removes fine detail.

16.2 Gaussian Blur

Uses a weighted kernel based on the Gaussian function:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right).$$

It produces smoother, more natural blurring and preserves edges better than average blurring.

Figure 9: Gaussian Blur

16.3 Sobel Edge Detection

Edge detection aims to identify points in an image where the intensity changes abruptly. Such changes usually correspond to object boundaries, texture transitions, and structural features. One of the most widely used gradient-based edge detection methods is the **Sobel operator**, which estimates the first-order derivatives of the image intensity in both horizontal and vertical directions.

Unlike simple derivative filters, the Sobel operator combines differentiation with local smoothing, making it more robust to noise while still emphasizing edges.

16.3.1 Sobel Operator

The Sobel operator approximates these derivatives using discrete convolution masks. It uses two 3×3 kernels: one for the horizontal derivative (Sobel X) and one for the vertical derivative (Sobel Y).

16.3.2 Sobel X (Horizontal Gradient)

The Sobel X kernel emphasizes changes along the *horizontal direction*, making it sensitive to *vertical edges*:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Convolving the image with G_x computes an approximation of:

$$G_x(x, y) \approx \frac{\partial I}{\partial x}$$

16.3.3 Sobel Y (Vertical Gradient)

The Sobel Y kernel emphasizes changes along the *vertical direction*, making it sensitive to *horizontal edges*:

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Convolving the image with G_y computes an approximation of:

$$G_y(x, y) \approx \frac{\partial I}{\partial y}$$

Interpretation:

- Large values indicate strong horizontal edges.
- The top and bottom rows have opposite signs, measuring the difference between upper and lower neighborhoods.
- The larger central weights again provide smoothing in the orthogonal direction.

16.3.4 Edge Magnitude and Direction

After computing the horizontal and vertical gradients using Sobel X and Sobel Y, the overall edge strength at each pixel is obtained by:

$$G = \sqrt{G_x^2 + G_y^2}$$

The edge orientation is given by:

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

16.3.5 Why Sobel Works Well?

The Sobel operator combines:

- **Differentiation:** to detect intensity changes,
- **Smoothing:** to suppress noise via weighted averaging.

This makes Sobel more robust than simple finite difference operators while remaining computationally efficient.

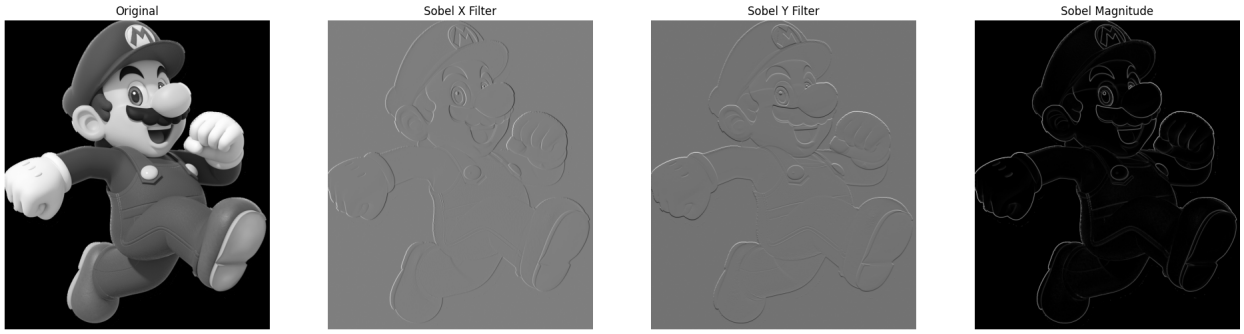


Figure 10: Sobel Edge Detection

16.4 Canny Edge Detection

Canny is a multi-stage algorithm: Gaussian smoothing, gradient computation, non-maximum suppression, double thresholding, and hysteresis-based edge tracking. It produces thin, well-localized, and noise-robust edges.

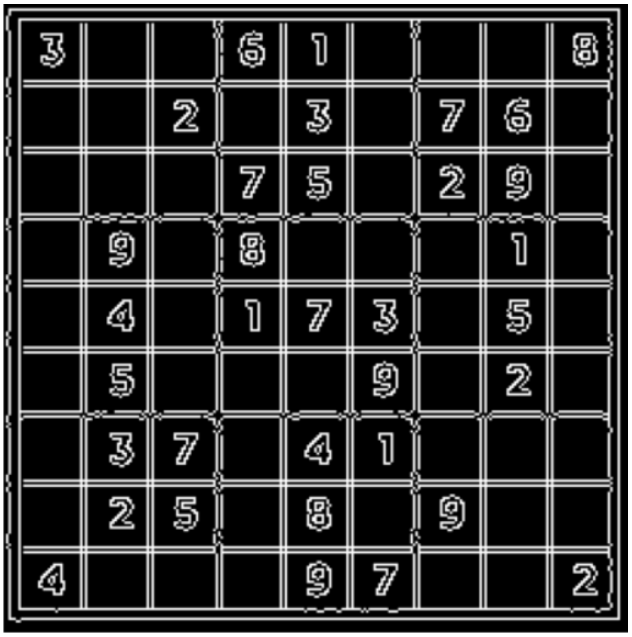


Figure 11: Canny Edge Map of a Sudoku Puzzle

16.5 Sharpening (Laplacian)

The Laplacian is a second-order derivative operator that highlights rapid intensity changes. Sharpening is performed by subtracting the Laplacian from the original image:

$$I_{\text{sharp}} = I - \alpha \nabla^2 I.$$

16.6 Unsharp Masking

First, a blurred image B is computed. A mask $M = I - B$ is formed and added back to the original:

$$I_{\text{sharp}} = I + kM.$$

This enhances edges while preserving smooth regions.

17 Binary Images

A binary image is an image in which each pixel can take only one of two possible values, typically 0 (background) and 1 (foreground), or equivalently black and white. Binary images

are usually obtained by applying a threshold to a grayscale image:

$$B(x,y) = \begin{cases} 1, & I(x,y) \geq T, \\ 0, & I(x,y) < T, \end{cases}$$

where T is a threshold value.

Binary images simplify image representation by separating objects from the background. This makes them especially useful for segmentation, shape analysis, object counting, and morphological processing.

18 Segmentation Using Binary Images

Segmentation refers to dividing an image into meaningful regions. Thresholding is one of the simplest segmentation techniques. By selecting an appropriate threshold T, pixels belonging to objects of interest are separated from the background.

Common approaches include:

- **Global Thresholding:** A single threshold for the entire image.
- **Adaptive Thresholding:** Threshold varies locally to handle uneven illumination.
- **Otsu's Method:** Automatically selects T by maximizing between-class variance.

After segmentation, each connected foreground region typically corresponds to one object that can be analyzed independently.

19 Geometric Properties of Binary Images

Once an object has been segmented, several geometric properties can be computed directly from the binary image:

- **Area:** Total number of foreground pixels.
- **Perimeter:** Length of the object boundary.
- **Centroid:** The geometric center of the object.
- **Bounding Box:** Smallest rectangle enclosing the object.
- **Aspect Ratio:** Ratio of width to height.

These features are essential in object classification, recognition, and tracking.

20 Iterative Modification of Binary Images

Binary images are often refined using iterative morphological operations:

- **Erosion:** Removes boundary pixels, shrinking objects.
- **Dilation:** Adds pixels to boundaries, expanding objects.
- **Opening:** Erosion followed by dilation; removes noise.
- **Closing:** Dilation followed by erosion; fills holes.

Such operations improve segmentation quality and produce clean object shapes for further analysis.

21 Flat Regions, Edges, and Corners

Understanding the distinction between flat regions, edges, and corners is fundamental to feature detection in computer vision.

21.1 Flat Regions

In flat regions, pixel intensities change very little in all directions:

$$I_x \approx 0, \quad I_y \approx 0$$

These regions contain little structural information and are generally ignored by feature detectors.

21.2 Edges

Edges occur where intensity changes sharply in one direction but remains nearly constant in the perpendicular direction:

$$I_x \gg 0, \quad I_y \approx 0 \quad \text{or} \quad I_y \gg 0, \quad I_x \approx 0$$

Edges correspond to object boundaries and are useful for segmentation but are less distinctive for precise localization.

21.3 Corners

Corners are points where intensity changes significantly in both directions:

$$I_x \gg 0, \quad I_y \gg 0$$

They represent the intersection of two or more edges and are highly distinctive and stable for tracking and matching.

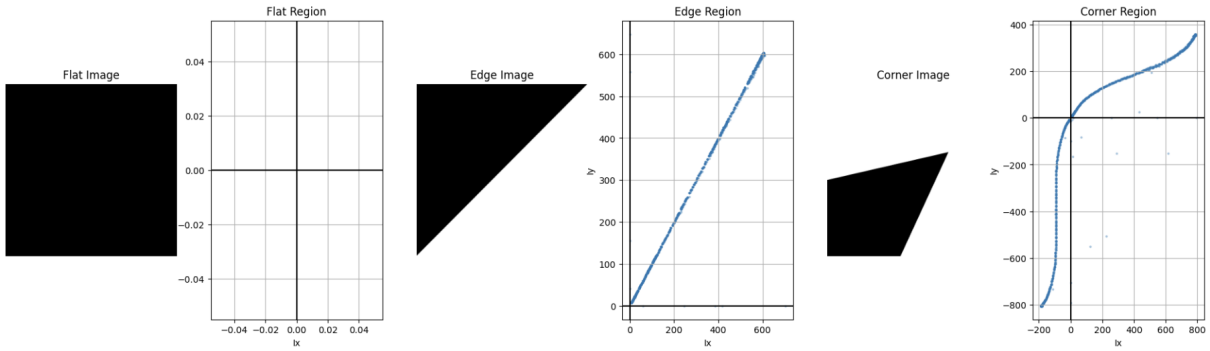


Figure 12: Scatter plot for flat, edge and corner

21.4 Mathematical Interpretation

Using the structure tensor

$$M = \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix},$$

- Flat region: both eigenvalues small.
- Edge: one eigenvalue large, one small.
- Corner: both eigenvalues large.

This formulation provides the foundation for corner detectors such as the Harris detector.

22 Harris Corner Detection

Harris Corner Detection is a classical feature detection method used to identify *interest points*, commonly referred to as corners, in an image. A corner is defined as a point where the image intensity varies significantly in both horizontal and vertical directions. Such points

are highly distinctive, making them reliable for tasks such as image matching, tracking, registration, and object recognition.

Unlike edges, which exhibit strong intensity variation in only one direction, corners exhibit strong variation in two orthogonal directions. This makes them stable under small changes in viewpoint, illumination, and noise.

22.1 Motivation

Local image regions can be characterized according to how intensity changes within a small neighborhood. In flat regions, the intensity remains almost constant in all directions. Along edges, intensity changes significantly in one direction but remains nearly constant in the orthogonal direction. At corners, however, intensity changes strongly in all directions. This distinct behavior allows corners to be used as reliable and repeatable feature points.

Because corners are localized, geometrically meaningful, and robust to moderate image distortions, they form the backbone of many feature-based vision algorithms.

22.2 Basic Idea

The central idea of the Harris detector is to measure how much the appearance of a small window around a pixel changes when the window is shifted in different directions. Let W be a small window centered at pixel (x, y). If the window is shifted by (u, v), the change in appearance is measured using the sum of squared differences (SSD):

$$E(u, v) = \sum_{(x,y)\in W} [I(x + u, y + v) - I(x, y)]^2$$

If the value of $E(u, v)$ is small for all shifts, the region is flat. If $E(u, v)$ is large only for shifts in one direction, the region corresponds to an edge. If $E(u, v)$ is large for shifts in all directions, the region corresponds to a corner.

22.3 Mathematical Formulation

To obtain a computationally efficient formulation, the image intensity function is approximated using a first-order Taylor expansion:

$$I(x + u, y + v) \approx I(x, y) + I_x u + I_y v$$

Substituting this approximation into the SSD expression gives:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

where M is known as the second-moment matrix or structure tensor:

$$M = \sum_{(x,y)\in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

In practice, the summation is weighted using a Gaussian window to emphasize pixels near the center of the window and reduce sensitivity to noise.

The matrix M captures the local gradient structure of the image. Its eigenvalues and eigenvectors describe how intensity varies in different directions within the window.

22.4 Eigenvalue Interpretation

Let λ_1 and λ_2 be the eigenvalues of the matrix M. Their magnitudes indicate the degree of intensity variation along two orthogonal directions. If both eigenvalues are small, the region is flat. If one eigenvalue is large and the other is small, the region corresponds to an edge. If both eigenvalues are large, the region exhibits strong variation in all directions and is therefore classified as a corner.

This interpretation provides a geometric understanding of why corners are detected at points where local image structure is rich in multiple directions.

22.5 Harris Response Function

Direct computation of eigenvalues at every pixel is computationally expensive. Instead, Harris proposed a response function that combines the determinant and trace of the matrix M:

$$R = \det(M) - k\,(\text{trace}(M))^2$$

where:

$$\det(M) = \lambda_1 \lambda_2, \quad \text{trace}(M) = \lambda_1 + \lambda_2$$

The constant k is an empirical sensitivity factor, typically chosen in the range 0.04 to 0.06.

A large positive value of R indicates the presence of a corner, a negative value indicates an edge, and values near zero correspond to flat regions.

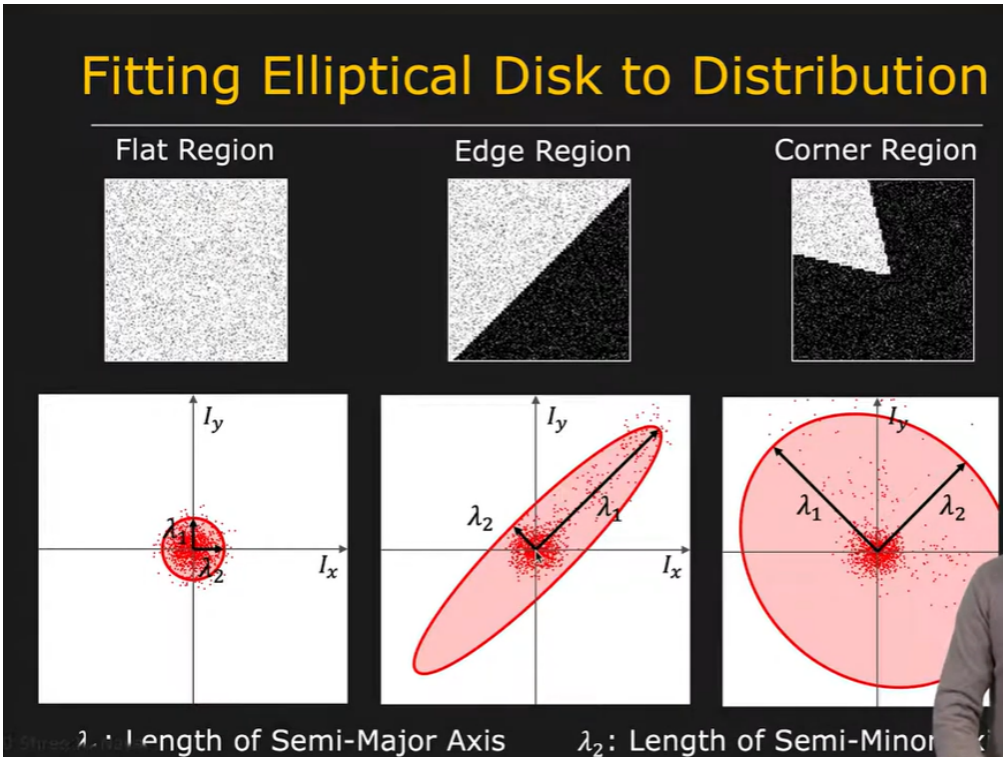


Figure 13: Harris Corner Detection Visualization

22.6 Algorithm

The Harris Corner Detection algorithm proceeds as follows. First, image gradients I_x and I_y are computed using derivative filters. The products I_x^2, I_y^2, and $I_x I_y$ are then smoothed using a Gaussian filter. For each pixel, the matrix M is formed from these smoothed values, and the Harris response R is computed. Finally, pixels with R above a predefined threshold are retained, and non-maximum suppression is applied to obtain well-localized corner points.

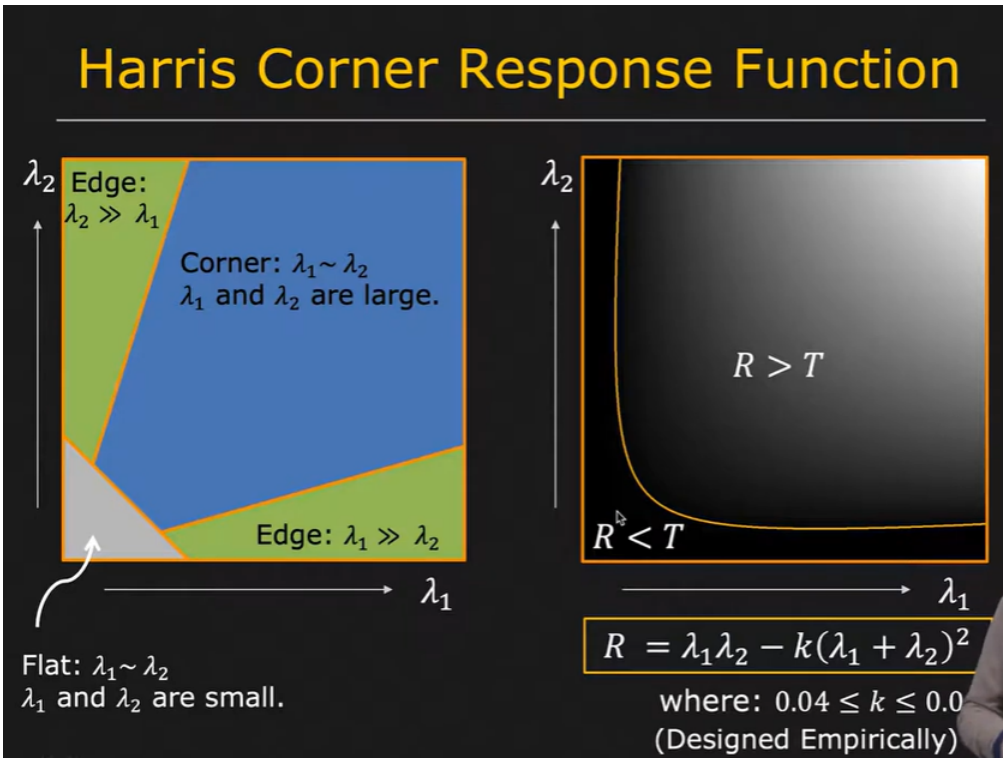


Figure 14: Harris Corner Detection Process

22.7 Properties

Harris corner detection is invariant to image rotation and robust to moderate levels of noise due to the use of gradient averaging. It is also computationally efficient and well-suited for real-time applications. However, it is not inherently scale invariant, meaning that corners detected at one image scale may not be detected at another. Additionally, strong illumination changes can affect gradient magnitudes and hence the detector's response.

23 Hough Transform

The Hough Transform is a global feature extraction technique used to detect parametric geometric structures such as lines, circles, and ellipses. Unlike local edge-based methods, the Hough Transform is robust to noise, missing data, and fragmented edges because it operates by accumulating evidence in a parameter space.

23.1 General Method

The Hough Transform converts the problem of detecting shapes in the image domain into a search for peaks in a parameter domain. After extracting edge points from the image, each edge pixel votes for all parameter values that are consistent with its possible membership in a geometric shape. An accumulator array records these votes, and local maxima in this array correspond to detected shapes.

23.2 Line Detection

23.2.1 Why Not Use $y = mx + c$

The slope-intercept form

$$y = mx + c$$

is unsuitable for line detection because vertical lines correspond to infinite slopes and the parameter space becomes unbounded. This leads to numerical instability and poor discretization of the accumulator.

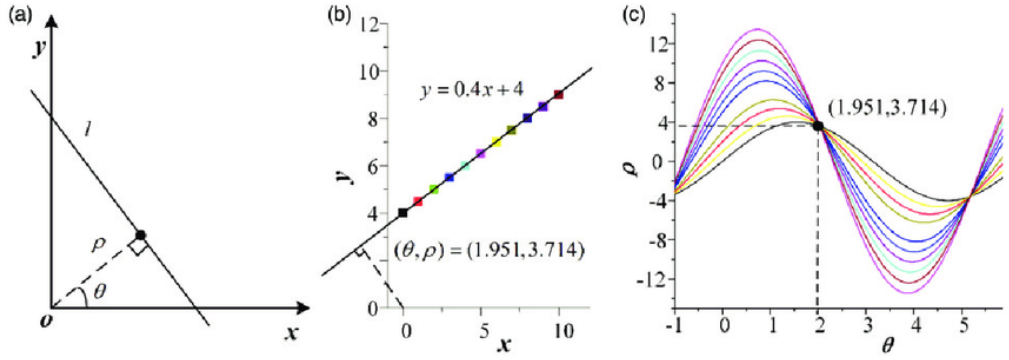


Figure 15: Line Representation Issues

23.2.2 Polar Representation

To avoid this issue, lines are represented in polar form:

$$\rho = x \cos\theta + y \sin\theta$$

where ρ is the perpendicular distance from the origin to the line and θ is the angle of the normal vector. Each edge pixel (x, y) corresponds to a sinusoidal curve in (ρ, θ) space. When multiple curves intersect at a common point in parameter space, that intersection represents the parameters of a line in the image.

23.3 Circle Detection with Fixed Radius

A circle with known radius r is described by:

$$(x-a)^2 + (y-b)^2 = r^2$$

where (a, b) denotes the center. In this case, the parameter space is two-dimensional. Each edge pixel votes for possible center locations lying on a circle of radius r centered at (x, y). Peaks in the accumulator correspond to detected circle centers.

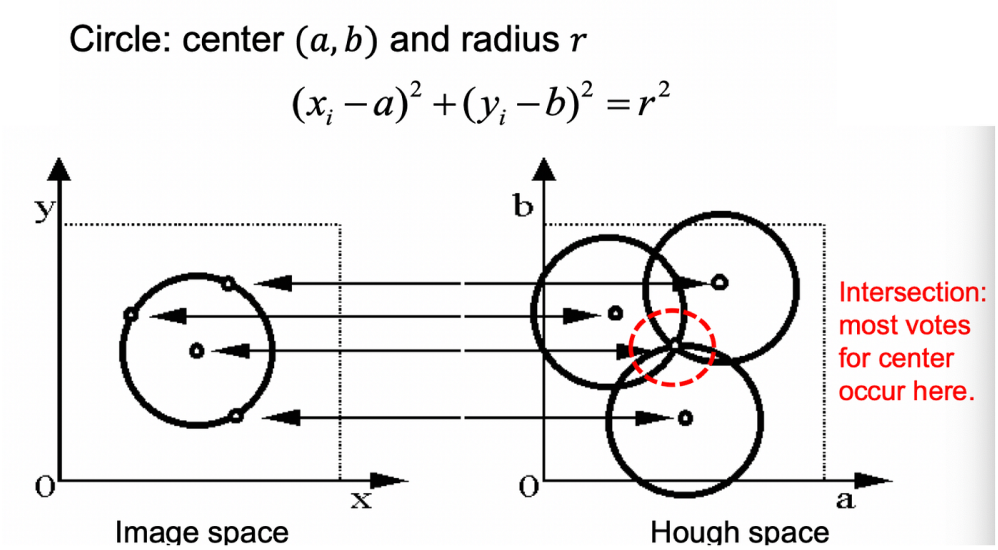


Figure 16: Circle Detection with Fixed Radius

23.3.1 Using Gradient Direction

For a true circular edge, the image gradient at a boundary pixel points radially toward the center of the circle. If θ_g denotes the gradient direction at (x, y), the center can be directly estimated as:

$$a = x - r\cos\theta_g, \quad b = y - r\sin\theta_g$$

This restricts voting to a single location per edge pixel rather than an entire circle in parameter space. Consequently, computational cost is reduced, accumulator peaks become sharper, and robustness to noise is improved.

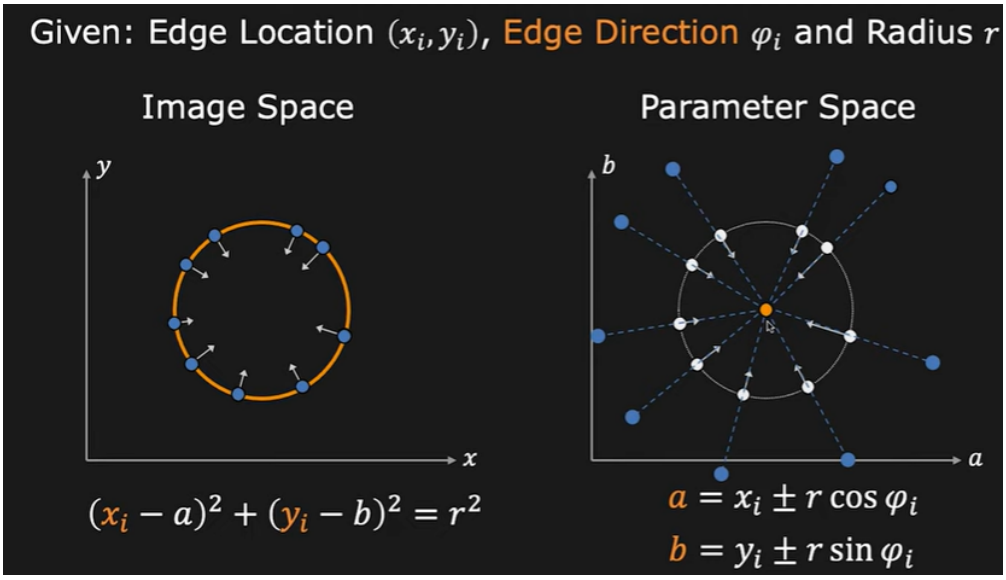


Figure 17: Circle Detection Using Gradient Direction

23.4 Circle Detection with Unknown Radius

When the radius is unknown, the circle equation introduces a third parameter:

$$(x - a)^2 + (y - b)^2 = r^2$$

The accumulator space becomes three-dimensional (a, b, r), significantly increasing computational complexity and memory requirements.

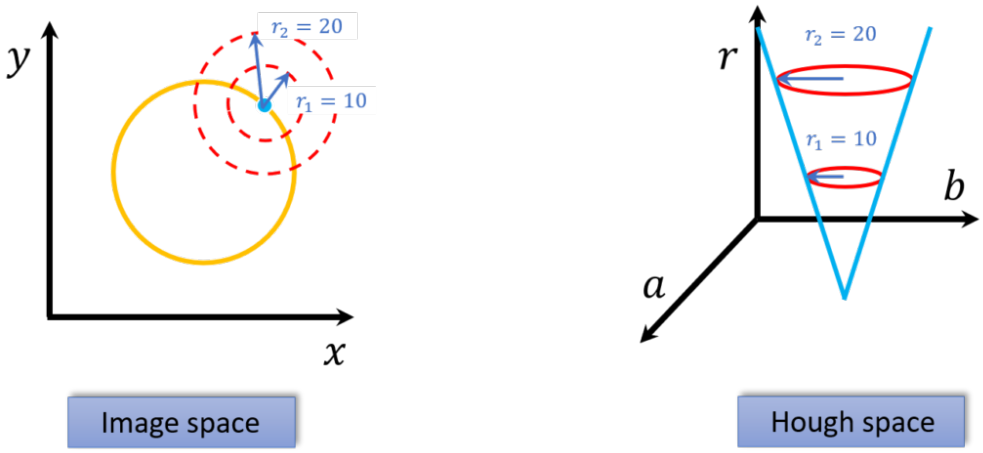


Figure 18: Circle Detection with Unknown Radius

23.4.1 Using Gradient Information

Even in the unknown-radius case, gradient direction can be used to restrict voting to plausible centers:

$$a = x - r\cos\theta_g, \quad b = y - r\sin\theta_g$$

This reduces the search space and improves detection efficiency.

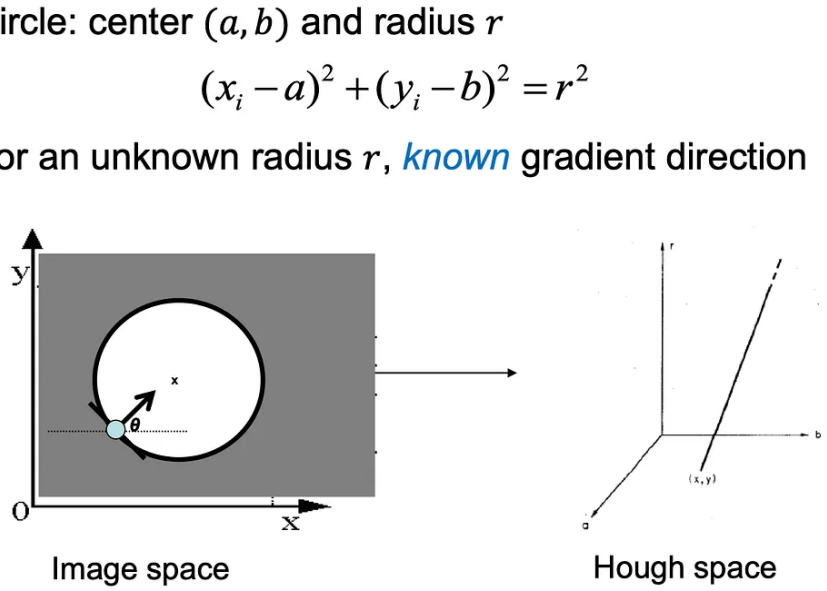


Figure 19: Circle Detection Using Gradient Information

23.5 Generalized Hough Transform (GHT)

The Generalized Hough Transform extends the original framework to detect arbitrary shapes that cannot be described by explicit analytical equations. Instead of using a parametric model, the shape is represented using a lookup structure known as a Φ-table.

23.5.1 The Φ-Table (Model Description)

A reference point (x_c, y_c) is selected within or near the object. For each boundary point (x_i, y_i) on the model shape, the edge orientation Φ_i is computed, and the vector displacement from the boundary point to the reference point is stored. The Φ-table indexes these displacement vectors by edge orientation.

Edge Direction	$\vec{r} = (r, \alpha)$
ϕ_1	$\vec{r}_1^{\,1}, \vec{r}_2^{\,1}, \vec{r}_3^{\,1}$
ϕ_2	$\vec{r}_1^{\,2}, \vec{r}_2^{\,2}$
$\vdots$	$\vdots$
ϕ_n	$\vec{r}_1^{\,n}, \vec{r}_2^{\,n}, \vec{r}_3^{\,n}, \vec{r}_4^{\,n}$

Figure 20: Φ-Table for Generalized Hough Transform

23.5.2 Detection Algorithm

During detection, an accumulator array $A(x_c, y_c)$ is initialized. For each edge pixel in the target image, its edge orientation is computed and used to index the Φ-table. Each corresponding displacement vector predicts a candidate reference point:

$$x_c = x_i + r\cos\alpha, \quad y_c = y_i + r\sin\alpha$$

The accumulator is incremented at this location. After processing all edge pixels, local maxima in the accumulator indicate likely object locations.

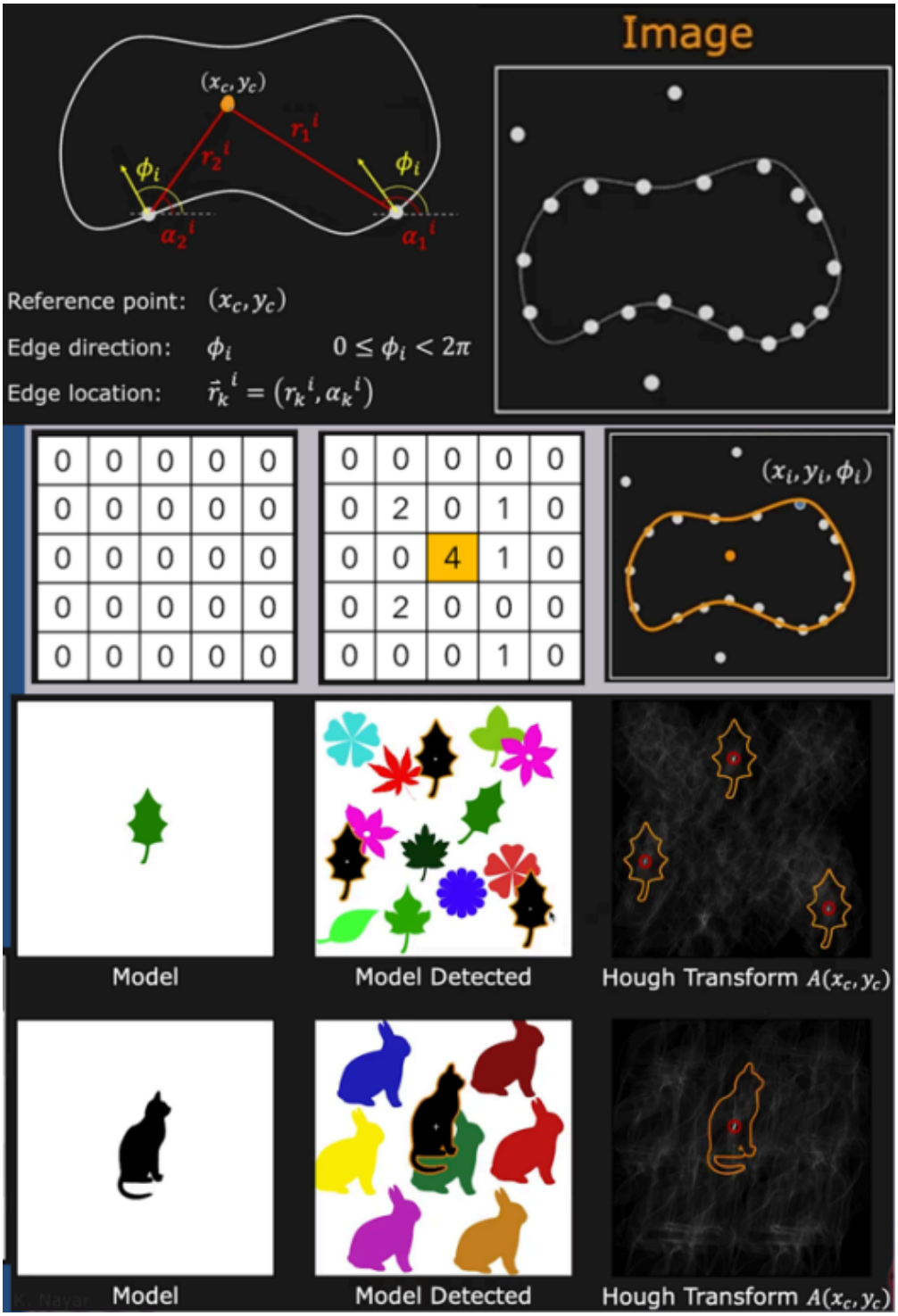


Figure 21: GHT Detection Algorithm

23.5.3 Handling Scale and Rotation

To detect objects under varying scale and orientation, additional parameters are introduced:

$$x_c = x_i + r \cdot s \cdot \cos(\alpha + \theta), \quad y_c = y_i + r \cdot s \cdot \sin(\alpha + \theta)$$

The accumulator becomes higher dimensional, enabling detection under geometric transformations at the cost of increased computational complexity.

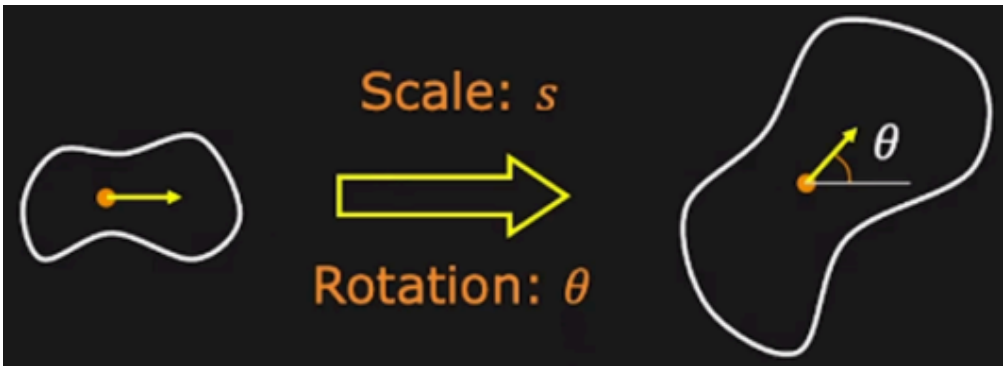


Figure 22: GHT with Scale and Rotation

23.6 Ellipse Detection

An ellipse is parameterized by five values: center (a, b), major and minor axes (α, β), and orientation θ:

$$\frac{(x-a)^2}{\alpha^2} + \frac{(y-b)^2}{\beta^2} = 1$$

Detecting ellipses using the Hough Transform requires a five-dimensional parameter space $(a, b, \alpha, \beta, \theta)$, which is computationally expensive. In practice, constraints from gradient orientation and geometric heuristics are employed to reduce the dimensionality of the search.

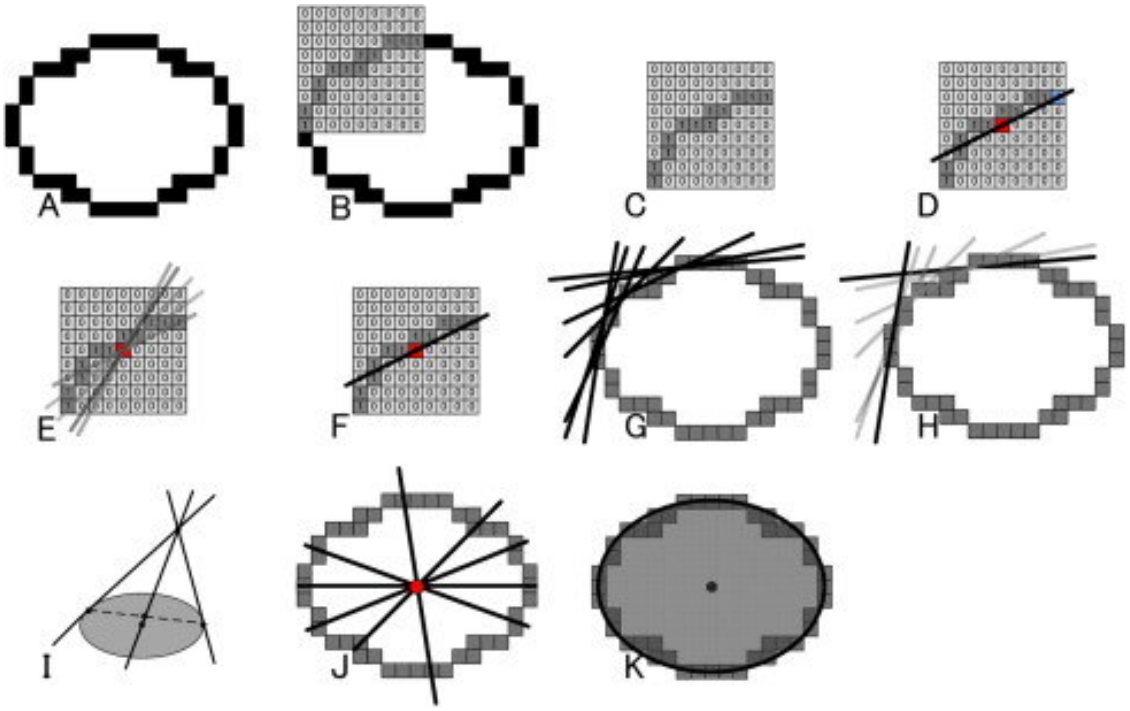


Figure 23: Ellipse Detection

24 Feature Extraction in Computer Vision

In computer vision, raw pixel intensities are rarely sufficient for effective recognition or classification. Instead, images are transformed into structured numerical representations known as *features*. These features capture essential visual properties such as color, shape, and texture in a compact and interpretable form. This section presents three important categories of features: color features, shape features, and texture features. Particular emphasis is placed on Hu moments for shape description and Local Binary Patterns (LBP) for texture representation.

25 Color Features

Color features describe the chromatic characteristics of objects in an image. Rather than using raw RGB values, color is often analyzed in perceptually meaningful spaces such as HSV (Hue, Saturation, Value). A compact yet expressive color representation is obtained by computing statistical measures—specifically, the mean and standard deviation—of each HSV channel.

25.1 Mean Hue

Hue represents the dominant color type and corresponds to angular position on the color wheel. The mean hue value provides a summary of the overall color identity of an object. Objects with distinct color characteristics tend to cluster around specific hue ranges, making this feature highly discriminative for color-based classification.

25.2 Standard Deviation of Hue

While mean hue indicates the dominant color, the standard deviation of hue measures how uniformly that color is distributed across the object. A low standard deviation implies a consistent surface color, whereas a high standard deviation indicates variations in pigmentation or surface patterns. This feature enables discrimination between objects that share similar average color but differ in color uniformity.

25.3 Mean Saturation

Saturation measures the purity or vividness of color. The mean saturation value reflects whether an object appears rich and vibrant or dull and washed out. This property is useful for distinguishing visually similar objects that differ in surface quality or material characteristics.

25.4 Standard Deviation of Saturation

The standard deviation of saturation captures the degree of variation in color intensity across an object. Objects with textured or non-uniform surfaces tend to exhibit greater saturation variation, while smooth surfaces show relatively constant saturation. This feature provides information about surface consistency.

25.5 Mean Value (Brightness)

The Value channel represents luminance or brightness. The mean value indicates how light or dark an object appears. Brightness can be a significant cue in separating objects with similar color but different reflectance or illumination properties.

25.6 Standard Deviation of Value

The standard deviation of the Value channel quantifies the spread of light and shadow over the object. Objects with specular highlights, rough surfaces, or strong shading effects

typically exhibit higher brightness variation, while uniformly illuminated objects exhibit lower variation.

Together, these six color features—mean and standard deviation of Hue, Saturation, and Value—form a concise yet informative representation of chromatic appearance.

26 Shape Features

While color describes appearance, shape features encode the geometric structure of objects. Shape features are generally extracted from object contours or edges and are designed to be invariant to scale, translation, and minor distortions. The following geometric descriptors provide a robust representation of object form.

26.1 Area Ratio

Area ratio measures the proportion of the image occupied by the object. Let A_o be the area of the object (number of foreground pixels) and A_I be the total image area. Then:

$$\text{Area Ratio} = \frac{A_o}{A_I}$$

By normalizing object area with respect to the total image area, this feature becomes independent of absolute scale.

26.2 Aspect Ratio

Aspect ratio is defined as the ratio of width to height of the object's bounding box. Let w be the width and h be the height of the bounding box:

$$\text{Aspect Ratio} = \frac{w}{h}$$

It describes the degree of elongation of an object. Elongated objects yield high aspect ratios, whereas compact or round objects yield ratios close to one.

26.3 Solidity

Solidity is the ratio between the area of an object and the area of its convex hull. Let A_o be the object area and A_c be the area of the convex hull:

$$\text{Solidity} = \frac{A_o}{A_c}$$

It measures how completely an object fills its convex envelope. Smooth, compact objects exhibit high solidity, while objects with indentations or concave boundaries exhibit lower solidity.

26.4 Circularity

Circularity quantifies how closely the shape of an object resembles a circle. It is defined as:

$$\text{Circularity} = \frac{4\pi A_o}{P^2}$$

where A_o is the object area and P is the perimeter of the object. A perfect circle attains the maximum value of 1, while elongated or irregular shapes yield lower values.

Collectively, these geometric features provide a compact description of object form that complements color-based information.

27 Hu Moments: Invariant Shape Descriptors

Image moments are numerical measures that describe the spatial distribution of pixel intensity within an object. By centering moments at the object's centroid and normalizing them with respect to scale and rotation, a set of invariant descriptors is obtained. These descriptors, known as *Hu moments*, remain unchanged under translation, scaling, and rotation.

Seven Hu moments are defined, denoted as ϕ_1 through ϕ_7. Each moment captures a distinct geometric characteristic of the object's shape.

27.1 List of the Seven Hu Moments

- ϕ_1: Overall shape spread
- ϕ_2: Shape variance
- ϕ_3: Skewness or asymmetry
- ϕ_4: Higher-order asymmetry
- ϕ_5: Interaction between orientation and skew
- ϕ_6: Shape complexity
- ϕ_7: Sensitivity to mirror reflection

Although all seven moments encode geometric information, lower-order moments are generally more stable and less sensitive to noise. Therefore, practical systems often rely primarily on the first two moments.

27.2 Interpretation of the First Two Hu Moments

27.2.1 ϕ_1: Overall Shape Spread

The first Hu moment measures how pixel intensities are distributed around the centroid of the object. It reflects the overall spatial extent of the shape.

27.2.2 ϕ_2: Shape Variance

The second Hu moment measures how much the distribution of pixels deviates from uniformity. While ϕ_1 captures the magnitude of spread, ϕ_2 captures the degree of variation in that spread.

Together, ϕ_1 and ϕ_2 provide a compact yet powerful summary of object geometry.

28 Texture Features: Local Binary Patterns (LBP)

Texture features describe local spatial patterns that are not captured by color or shape alone. One of the most widely used texture descriptors is the *Local Binary Pattern* (LBP).

The core idea of LBP is to characterize the local structure of an image by comparing each pixel with its neighboring pixels.

28.1 Local Pattern Encoding

For a center pixel with intensity I_c and P neighboring pixels with intensities I_p, the LBP code is defined as:

$$\mathbf{LBP} = \sum_{p=0}^{P-1} s(I_p - I_c)\, 2^p$$

where

$$s(x) = \begin{cases} 1, & x \geq 0, \\ 0, & x < 0. \end{cases}$$

Each pixel is thus represented by a binary number that encodes the relative brightness of its neighborhood. This encoding captures micro-patterns such as edges, corners, flat regions, and spot-like textures.

28.2 Key Properties

Local Binary Patterns possess several important properties:

- **Illumination Robustness:** Since LBP relies on relative comparisons rather than absolute intensity values, it is largely invariant to uniform changes in lighting.
- **Rotation Invariance (Optional):** Rotation-invariant LBP is obtained by circularly rotating the binary pattern and selecting the minimum value:

$$\text{LBP}_{ri} = \min\{\text{ROR}(\text{LBP}, k)\}, \quad k = 0, 1, \ldots, P-1$$

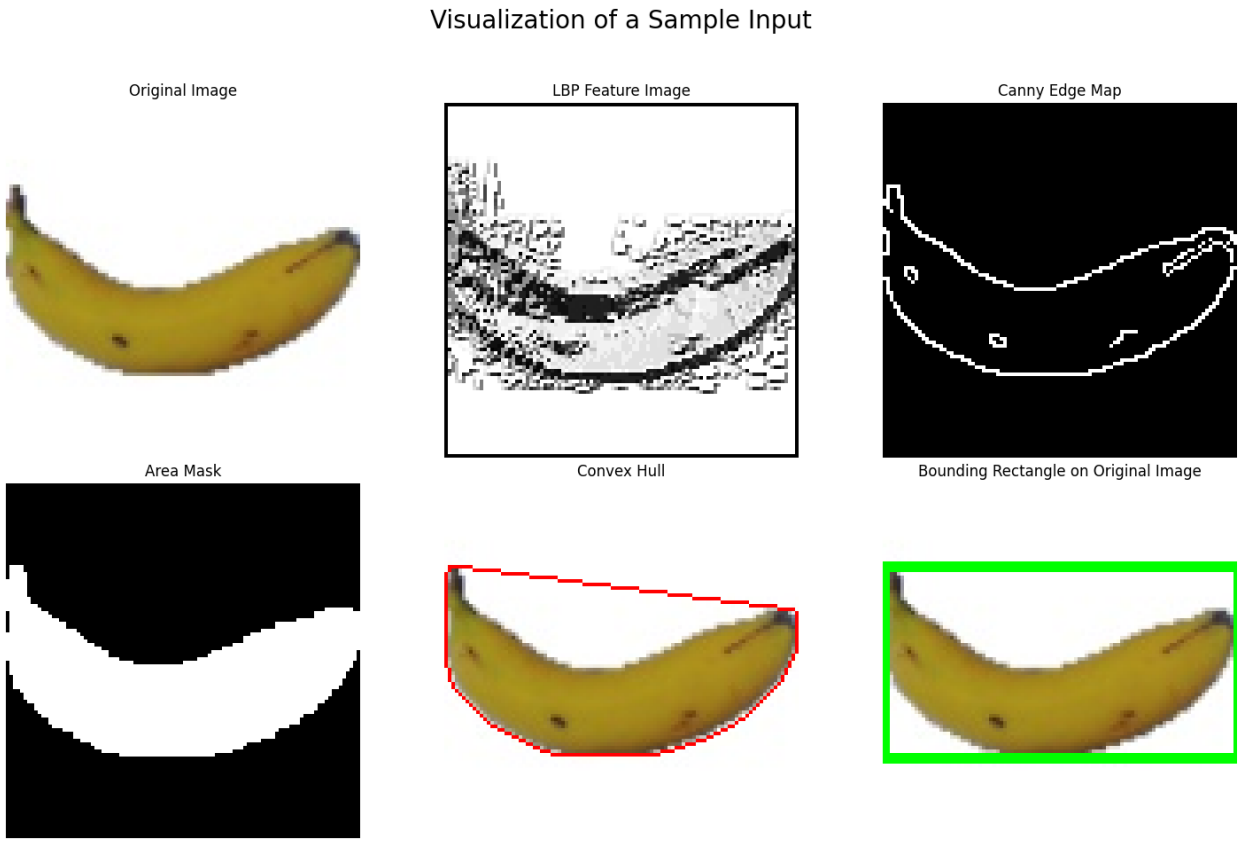


Figure 24: Local Binary Pattern (LBP) Features

28.3 LBP Histograms

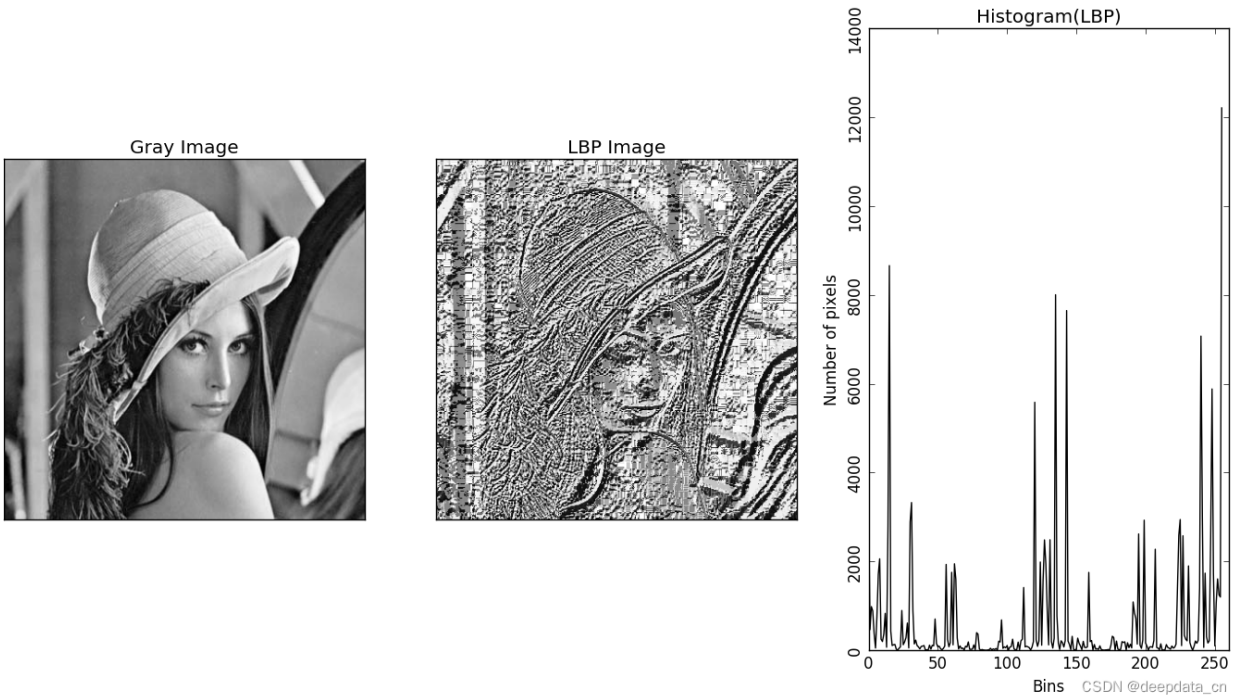


Figure 25: LBP Histogram Representation

29 Supervised Learning: Introduction

Machine Learning (ML) enables computational systems to learn patterns from data and make predictions or decisions without explicit rule-based programming. The goal of learning is not merely to fit the observed data, but to generalize well to unseen examples. Based on the availability of labeled outputs, machine learning algorithms are broadly categorized into supervised and unsupervised learning.

This section presents a detailed discussion of both learning paradigms, with particular emphasis on linear and logistic regression models, data preprocessing through normalization, regularization techniques, model evaluation metrics, hyperparameters, and the fundamental challenges of overfitting and underfitting.

30 Supervised Learning

Supervised learning algorithms operate on labeled datasets consisting of input–output pairs $(\mathbf{x}, y)$. The objective is to learn a mapping

$$f(\mathbf{x}) \approx y$$

that minimizes prediction error on unseen data.

Supervised learning is widely used in regression tasks, where outputs are continuous, and classification tasks, where outputs are discrete class labels. The availability of labels allows the model to compute an explicit loss function, which guides the learning process.

30.1 Training, Validation, and Testing

To ensure reliable performance estimation, datasets are typically divided into training, validation, and test sets. The training set is used to learn model parameters, while the validation set is used for hyperparameter tuning and model selection. The test set is reserved for final evaluation.

Cross-validation techniques, such as k-fold cross-validation, are commonly employed to reduce variance in performance estimates. In practice, machine learning libraries such as `scikit-learn` provide standardized tools for data splitting and cross-validation.

31 Data Preprocessing and Z-Score Normalization

Before training a model, feature preprocessing plays a critical role in ensuring stable and efficient learning. When features have different scales, models that rely on gradient-based optimization or regularization can behave poorly.

Z-score normalization, also known as standardization, rescales features to have zero mean and unit variance:

$$x' = \frac{x - \mu}{\sigma} \tag{1}$$

where μ is the mean and σ is the standard deviation of the feature.

Z-score normalization is particularly important for linear and logistic regression when regularization is applied, as it ensures that all features are penalized equally. In `scikit-learn`, this is commonly implemented using `StandardScaler`.

32 Linear Regression

Linear regression is a foundational supervised learning model that assumes a linear relationship between input features and the output variable.

32.1 Model Formulation

$$\hat{y} = \beta_0 + \sum_{i=1}^{n} \beta_i x_i \tag{2}$$

In matrix form:

$$\hat{y} = \mathbf{X}\boldsymbol{\beta} \tag{3}$$

32.2 Loss Function

The model parameters are learned by minimizing the Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{4}$$

33 Logistic Regression

Logistic regression is a supervised learning algorithm used for binary classification problems. It models the conditional probability of class membership using a logistic (sigmoid) function.

33.1 Model Formulation

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}} \tag{5}$$

33.2 Loss Function

The Binary Cross-Entropy loss is minimized during training:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right] \tag{6}$$

34 Overfitting and Underfitting

A fundamental challenge in machine learning is achieving good generalization performance.

Underfitting occurs when a model is too simple to capture the underlying structure of the data. Such models exhibit high bias and perform poorly on both training and test sets.

Overfitting occurs when a model learns noise and spurious patterns specific to the training data. Overfitted models achieve low training error but suffer from high test error, indicating poor generalization. This behavior is associated with high variance.

Balancing bias and variance is essential for building reliable predictive models.

35 Regularization Techniques

Regularization mitigates overfitting by introducing a penalty on model complexity. This is achieved by adding a regularization term to the loss function.

35.1 L2 Regularization (Ridge Regression)

L2 regularization penalizes the squared magnitude of the parameters:

$$\mathcal{L}_{\text{ridge}} = \mathcal{L}_{\text{MSE}} + \lambda \sum_{j=1}^{n} \beta_j^2 \tag{7}$$

L2 regularization shrinks model weights smoothly and is effective when features are correlated. It improves numerical stability and reduces sensitivity to noise.

35.2 L1 Regularization (Lasso Regression)

L1 regularization penalizes the absolute value of the parameters:

$$\mathcal{L}_{\text{lasso}} = \mathcal{L}_{\text{MSE}} + \lambda \sum_{j=1}^{n} |\beta_j| \tag{8}$$

L1 regularization encourages sparsity, often driving some coefficients exactly to zero, thereby performing implicit feature selection.

35.3 When and Why to Use L1 vs L2

L1 regularization is preferred when the dataset contains many irrelevant or redundant features and interpretability is important. L2 regularization is preferred when all features are informative and multicollinearity is present.

36 Hyperparameters and Learning Rate

In machine learning, model parameters and hyperparameters play fundamentally different roles. Model parameters, such as weights and biases, are learned directly from the training data by minimizing a loss function. In contrast, *hyperparameters* are configuration variables that are set prior to training and control the learning process itself.

Common hyperparameters include the learning rate, regularization strength, number of training iterations, batch size, and choice of optimization algorithm. Although hyperparameters are not learned from data, they strongly influence convergence behavior, training stability, and generalization performance.

36.1 Learning Rate

The learning rate, typically denoted by η, is one of the most critical hyperparameters in optimization algorithms such as gradient descent. It controls the step size taken during parameter updates:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \tag{9}$$

where θ represents the model parameters and $\mathcal{L}$ is the loss function.

A learning rate that is too large can cause the optimization process to diverge or oscillate, preventing convergence. Conversely, a learning rate that is too small results in extremely slow convergence and may cause the optimizer to become trapped in suboptimal solutions.

36.2 Effect of Learning Rate on Training

The choice of learning rate directly affects both training speed and model performance. A high learning rate leads to faster initial progress but increases the risk of overshooting the minimum. A low learning rate provides more stable updates but requires significantly more iterations to converge.

In practice, learning rate selection represents a tradeoff between convergence speed and stability. This tradeoff is particularly important for models trained using gradient-based methods, such as logistic regression with iterative solvers.

36.3 Learning Rate and Feature Scaling

Feature scaling techniques such as Z-score normalization play an important role in effective learning rate selection. When features are on vastly different scales, gradient magnitudes can vary significantly across dimensions, making it difficult to choose a single learning rate that works well. Standardization ensures that all features contribute comparably to the gradient, allowing the learning rate to behave consistently.

36.4 Hyperparameter Tuning

Hyperparameters are typically selected using a validation set or cross-validation. Grid search and randomized search are commonly used techniques to explore hyperparameter values systematically. In the `scikit-learn` framework, tools such as `GridSearchCV` automate this process by evaluating model performance across different hyperparameter configurations.

Proper hyperparameter tuning is essential for balancing bias and variance and for achieving optimal generalization performance.

37 Model Evaluation Metrics

37.1 Regression Metrics

Mean Squared Error (MSE) and Mean Absolute Error (MAE) measure prediction error magnitude, while the coefficient of determination R^2 quantifies the proportion of variance explained by the model:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

37.2 Classification Metrics

For classification tasks, performance is evaluated using metrics derived from the confusion matrix, including accuracy, precision, recall, and F1-score. These metrics provide complementary insights, especially for imbalanced datasets.

38 Unsupervised Learning

Unsupervised learning deals with unlabeled data and aims to discover hidden patterns or structure within the dataset. Unlike supervised learning, there is no explicit error signal.

Clustering algorithms, such as k-means, group data points based on similarity, while dimensionality reduction techniques such as Principal Component Analysis (PCA) reduce feature dimensionality while preserving variance. Unsupervised learning is widely used in exploratory data analysis, anomaly detection, and data compression.

39 K-Means Clustering

39.1 Overview

K-Means is a centroid-based, iterative clustering algorithm that partitions a dataset into K disjoint clusters. Each cluster is represented by a centroid, defined as the arithmetic mean of all data points assigned to that cluster. The algorithm aims to produce compact and well-separated clusters by minimizing the variability of points within each cluster.

Despite its simplicity, K-Means captures several core ideas in unsupervised learning, including optimization under discrete assignments, the role of inductive assumptions, and the impact of initialization on learning outcomes.

39.2 Problem Formulation

Let the dataset be

$$X = \{x_1, x_2, \ldots, x_N\}, \quad x_i \in \mathbb{R}^D.$$

Given a predefined number of clusters K, the objective is to partition the dataset into K non-overlapping clusters $\{C_1, C_2, \ldots, C_K\}$ such that every data point belongs to exactly one cluster:

$$\bigcup_{k=1}^{K} C_k = X, \qquad C_i \cap C_j = \emptyset \text{ for } i \neq j.$$

Each cluster C_k is summarized by a centroid $\mu_k \in \mathbb{R}^D$, which represents the central tendency of the cluster.

39.3 Objective Function

The K-Means algorithm minimizes the *Within-Cluster Sum of Squares (WCSS)*, defined as

$$\min_{\{\mu_k\}_{k=1}^{K}} \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2. \tag{10}$$

This objective function measures the total squared Euclidean distance between each data point and the centroid of its assigned cluster. Minimizing this quantity encourages points within the same cluster to be close to each other, resulting in compact, spherical clusters.

39.4 Loss Function Interpretation

An equivalent formulation of the objective uses cluster assignment variables $z_i \in \{1, \ldots, K\}$:

$$\mathcal{L} = \sum_{i=1}^{N} \|x_i - \mu_{z_i}\|^2. \tag{11}$$

From an optimization perspective, this loss function is non-convex due to the discrete nature of the cluster assignments. As a result, K-Means is not guaranteed to find the global minimum, but instead converges to a locally optimal solution.

39.5 Algorithm

The K-Means algorithm proceeds by alternating between two steps:

1. **Assignment Step:** Each data point is assigned to the nearest centroid according to the Euclidean distance,
$$z_i = \arg\min_k \|x_i - \mu_k\|^2.$$
2. **Update Step:** Each centroid is recomputed as the mean of all points assigned to the corresponding cluster,
$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i.$$

These steps are repeated until convergence, which typically occurs when cluster assignments or centroid positions no longer change significantly.

39.6 Optimization Perspective

K-Means can be viewed as a form of coordinate descent on the objective function. During the assignment step, the loss is minimized with respect to the cluster assignments while keeping the centroids fixed. During the update step, the loss is minimized with respect to the centroids while keeping the assignments fixed.

Each iteration is guaranteed to reduce (or leave unchanged) the value of the objective function, ensuring monotonic convergence.

39.7 Convergence Properties

K-Means is guaranteed to converge in a finite number of iterations. However, due to the non-convexity of the objective function, the final solution depends strongly on the initialization of the centroids. Different initializations may lead to different local minima, motivating the use of multiple restarts or improved initialization schemes such as K-Means++.

39.8 Computational Complexity

The computational complexity of one iteration of K-Means is

$$O(NKD),$$

where N is the number of data points, K is the number of clusters, and D is the dimensionality of the data. This linear scaling makes K-Means suitable for large datasets when K and D are moderate.

39.9 Assumptions and Practical Considerations

K-Means relies on several implicit assumptions. It assumes that clusters are approximately spherical, of comparable size, and well separated under the Euclidean distance metric. Additionally, the algorithm is sensitive to feature scaling, making normalization or standardization a crucial preprocessing step.

39.10 Choosing the Number of Clusters

Selecting an appropriate value of K is a critical modeling decision. Common approaches include the elbow method, which examines the rate of decrease of WCSS as K increases, and the silhouette score, which measures how well-separated the resulting clusters are. In practice, domain knowledge often plays an important role in determining a meaningful number of clusters.

39.11 Limitations

Despite its popularity, K-Means has several limitations. It is sensitive to outliers, which can significantly distort centroid locations. The requirement to predefine K may be restrictive in exploratory settings, and the algorithm performs poorly when clusters are non-spherical, overlapping, or of highly unequal sizes.

39.12 Conclusion

K-Means clustering provides a simple yet powerful framework for partitioning data into meaningful groups. By minimizing within-cluster variance through an iterative optimization procedure, it captures essential structure in many real-world datasets. However, its effectiveness depends on the validity of its assumptions, careful preprocessing, and appropriate choice of hyperparameters.

40 Neural Networks: Foundations, Architectures, and Learning

Neural networks form the foundation of modern machine learning and deep learning systems. Inspired by biological neural systems, they provide a flexible framework for learning complex, non-linear relationships between inputs and outputs. Unlike traditional linear models, neural networks scale naturally with data and model capacity, making them suitable for tasks ranging from regression and classification to computer vision and natural language processing.

This section presents a structured discussion of neural networks, beginning with the perceptron model and extending to multi-layer architectures. Key components such as neurons, layers, activation functions, forward propagation, loss functions, and hyperparameter tuning are discussed in detail.

41 The Perceptron

The perceptron is the simplest form of a neural network and serves as a building block for more complex architectures. It models a binary classifier by computing a weighted sum of input features followed by a threshold-based decision.

Given an input vector $\mathbf{x} \in \mathbb{R}^n$, the perceptron computes

$$z = \mathbf{w}^T \mathbf{x} + b, \tag{12}$$

where $\mathbf{w}$ denotes the weight vector and b is the bias term. The output is obtained by applying an activation function $\phi(z)$.

Although a single perceptron can only learn linearly separable decision boundaries, it introduces the fundamental concepts of weights, bias, and activation that generalize to deeper neural networks.

42 Neurons and Layers

A neuron is the fundamental computational unit of a neural network. It performs an affine transformation of its inputs followed by a non-linear activation. Multiple neurons arranged together form a layer.

A neural network typically consists of:

- An input layer that receives raw features
- One or more hidden layers that learn intermediate representations
- An output layer that produces final predictions

Stacking multiple layers enables the network to learn hierarchical feature representations, where deeper layers capture increasingly abstract patterns in the data.

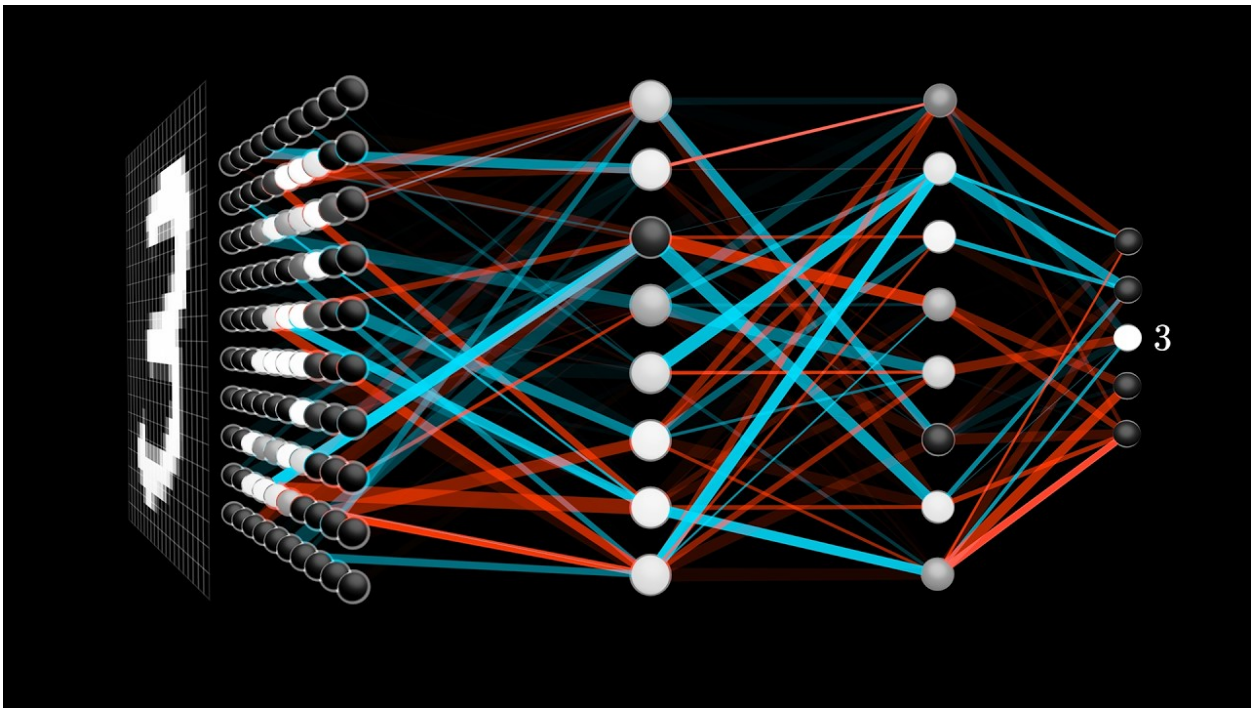


Figure 26: Neural Network Architecture

43 Weights and Biases

Weights determine the strength of influence each input feature has on a neuron's output. Biases allow neurons to shift activation thresholds independently of the input. Together, weights and biases define the expressive power of the network.

In practice, these parameters are initialized randomly and updated iteratively during training. Improper initialization can lead to slow convergence or unstable training, motivating careful initialization strategies in modern frameworks.

44 Activation Functions

Activation functions introduce non-linearity into neural networks, enabling them to learn complex decision boundaries and hierarchical representations. Without activation functions, a neural network—regardless of depth—would collapse into a linear model.

44.1 Sigmoid Activation

The sigmoid activation function maps real-valued inputs to the interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{13}$$

It is commonly used in the output layer for binary classification problems, where the output can be interpreted as a probability. However, sigmoid activations saturate for large positive or negative inputs, leading to vanishing gradients and slow convergence in deep networks.

44.2 Hyperbolic Tangent (Tanh)

The hyperbolic tangent function is defined as

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}. \tag{14}$$

Tanh outputs values in the range $(-1, 1)$ and is zero-centered, which often leads to faster optimization compared to sigmoid. Despite this advantage, tanh also suffers from vanishing gradients for large input magnitudes.

44.3 Rectified Linear Unit (ReLU)

The Rectified Linear Unit (ReLU) activation is given by

$$\mathrm{ReLU}(z) = \max(0, z). \tag{15}$$

ReLU has become the default choice for hidden layers in deep networks due to its computational simplicity and ability to mitigate the vanishing gradient problem. However, neurons can become permanently inactive if they consistently receive negative inputs, a phenomenon known as the *dying ReLU* problem.

44.4 Softmax Activation

The softmax function is primarily used in multi-class classification problems. Given a vector of logits $\mathbf{z}$, softmax produces a probability distribution:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}. \tag{16}$$

Softmax ensures that all output values lie in $(0, 1)$ and sum to one, making it suitable for modeling categorical probability distributions. It is typically paired with categorical cross-entropy loss during training.

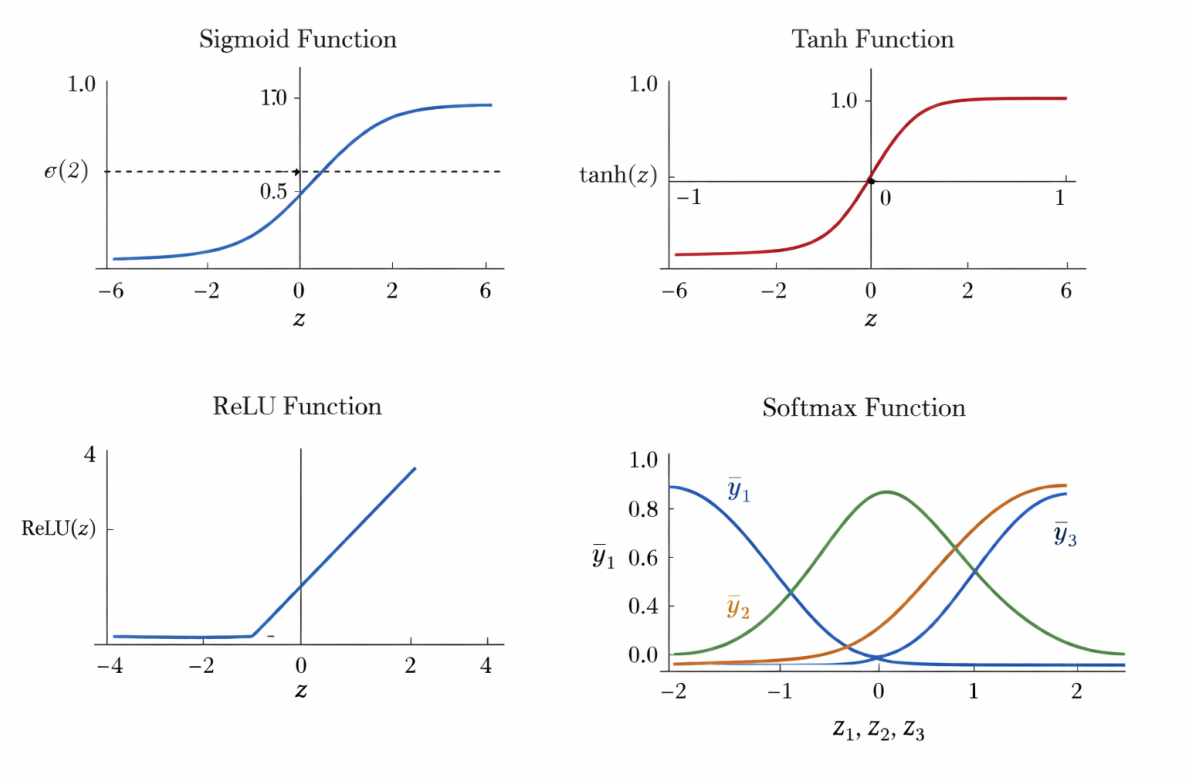


Figure 27: Common activation functions used in Neural Networks

45 Forward Pass

The forward pass refers to the process of computing the output of a neural network for a given input. Starting from the input layer, data flows through successive layers via affine transformations and activation functions.

For a layer l, the computation is given by

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \tag{17}$$

$$\mathbf{a}^{(l)} = \phi(\mathbf{z}^{(l)}), \tag{18}$$

where $\mathbf{a}^{(l)}$ denotes the activation of layer l.

The forward pass defines the model's predictions and directly influences the loss value.

46 Loss Functions

Loss functions quantify the discrepancy between predicted outputs and true targets.

46.1 Mean Squared Error (MSE)

Commonly used in regression tasks:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2. \tag{19}$$

MSE penalizes larger errors more heavily due to squaring.

46.2 Mean Absolute Error (MAE)

Defined as

$$\mathcal{L}_{\text{MAE}} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{20}$$

MAE is more robust to outliers but yields less smooth gradients.

46.3 Binary Cross-Entropy (BCE)

Used in binary classification:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]. \tag{21}$$

46.4 Categorical Cross-Entropy

Used in multi-class classification with softmax outputs:

$$\mathcal{L} = -\sum_{k} y_k \log(\hat{y}_k). \tag{22}$$

47 Backpropagation

Backpropagation is the fundamental algorithm used to train neural networks. It provides an efficient method for computing gradients of the loss function with respect to all trainable parameters in the network. These gradients are then used by optimization algorithms, such as gradient descent, to update weights and biases.

The core idea of backpropagation is the application of the chain rule from calculus to propagate error signals backward through the network layers. This allows each parameter to be adjusted in proportion to its contribution to the final prediction error.

47.1 Error Propagation

Given a loss function $\mathcal{L}$ and network output $\hat{y}$, backpropagation computes partial derivatives of the loss with respect to each parameter. For the output layer, the error signal is computed directly from the loss:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}}, \tag{23}$$

where $\mathbf{z}^{(L)}$ denotes the pre-activation values of the output layer.

For hidden layers, the error is propagated backward as

$$\delta^{(l)} = \left(\mathbf{W}^{(l+1)}\right)^T \delta^{(l+1)} \odot \phi'(\mathbf{z}^{(l)}), \tag{24}$$

where $\odot$ denotes element-wise multiplication and $\phi'(\cdot)$ is the derivative of the activation function.

47.2 Gradient Computation

Once the error terms are computed, gradients with respect to weights and biases are obtained as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} \left(\mathbf{a}^{(l-1)}\right)^T, \tag{25}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \tag{26}$$

These gradients quantify how each parameter influences the loss and guide the optimization process.

47.3 Parameter Updates

Using gradient descent, parameters are updated iteratively as:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}}, \tag{27}$$

$$\mathbf{b} \leftarrow \mathbf{b} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}}, \tag{28}$$

where η denotes the learning rate.

Through repeated forward and backward passes over the dataset, the network gradually minimizes the loss function and improves predictive performance.

47.4 Practical Considerations

Although backpropagation is computationally efficient, deep networks may suffer from issues such as vanishing or exploding gradients. Activation choices, proper weight initialization, normalization techniques, and adaptive optimizers are commonly employed to stabilize training.

Modern deep learning frameworks automatically compute gradients using automatic differentiation, allowing practitioners to focus on model design rather than manual gradient derivations.

48 Training and Hyperparameters

Training neural networks involves minimizing the loss function using gradient-based optimization techniques.

Key hyperparameters include:

- Learning rate: controls the step size of parameter updates
- Number of epochs: determines how many times the dataset is processed
- Batch size: controls the granularity of gradient updates

Choosing appropriate hyperparameters is critical for stable and efficient learning.

49 Convolutional Neural Networks: Theory, Architecture, and Representation Learning

Convolutional Neural Networks (CNNs) are a specialized class of deep learning models designed for processing structured grid-like data, particularly images. Unlike traditional fully connected neural networks, CNNs exploit the spatial structure of image data through local connectivity, weight sharing, and hierarchical feature learning.

CNNs have achieved remarkable success in computer vision applications such as image classification, object detection, facial recognition, and medical image analysis. Their effectiveness arises from their ability to learn meaningful spatial features directly from raw pixel data.

50 Motivation for Convolutional Neural Networks

Images exhibit strong local correlations: neighboring pixels are more closely related than distant ones. Fully connected neural networks ignore this spatial locality, resulting in a large number of parameters and increased risk of overfitting.

CNNs address these issues by:

- Connecting neurons only to local regions of the input
- Sharing filter weights across spatial locations
- Learning hierarchical feature representations

These properties significantly reduce model complexity while preserving expressive power.

51 Convolutional Layers

51.1 Convolution Operation

The convolutional layer is the core building block of a CNN. Given an input image $X \in \mathbb{R}^{H \times W}$ and a convolutional filter $K \in \mathbb{R}^{f \times f}$, the convolution operation produces a feature map Y:

$$Y(i,j) = \sum_{m=0}^{f-1} \sum_{n=0}^{f-1} X(i+m, j+n)\, K(m,n) \tag{29}$$

The filter slides across the input image and computes localized dot products. Each filter learns to detect a specific pattern, such as edges, corners, or textures.

51.2 Multiple Filters and Feature Maps

In practice, multiple filters are applied simultaneously to the same input. Each filter generates a separate feature map, and these maps are stacked to form the output volume of the convolutional layer.

Early convolutional layers typically capture low-level visual features, while deeper layers learn increasingly abstract and semantic representations.

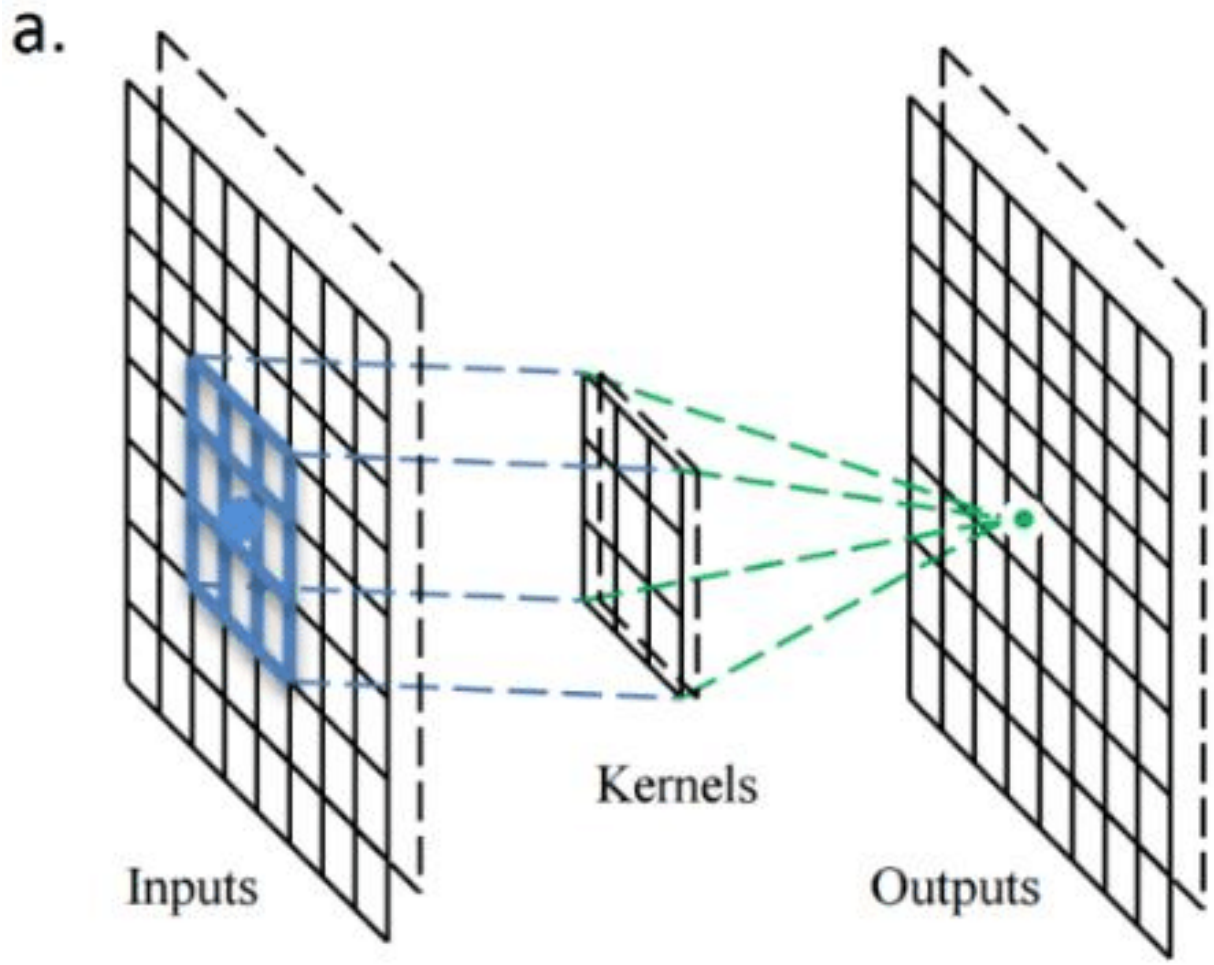


Figure 28: Convolution Operation

52 Stride and Padding

The stride determines how far the filter moves at each step during convolution. A larger stride reduces the spatial dimensions of the output feature map.

Padding involves adding extra pixels (usually zeros) around the input image. Padding helps control output size and ensures that edge information is preserved.

For an input of size $N \times N$, filter size F, padding P, and stride S, the output size is given by:

$$\text{Output size} = \frac{N - F + 2P}{S} + 1 \tag{30}$$

53 Pooling Layers

53.1 Purpose of Pooling

Pooling layers perform spatial downsampling of feature maps. They reduce dimensionality while retaining the most important information, leading to lower computational cost and improved generalization.

Pooling also provides a degree of translation invariance by reducing sensitivity to small spatial shifts in the input.

53.2 Max Pooling

Max pooling selects the maximum value within a local window:

$$Y(i,j) = \max_{(m,n)\in\Omega} X(i+m, j+n) \tag{31}$$

This operation preserves the strongest activations and is widely used in CNN architectures.

53.3 Average Pooling

Average pooling computes the mean value within the pooling region:

$$Y(i,j) = \frac{1}{|\Omega|} \sum_{(m,n)\in\Omega} X(i+m, j+n) \tag{32}$$

While less common than max pooling, average pooling is useful when global feature information is more important than local saliency.

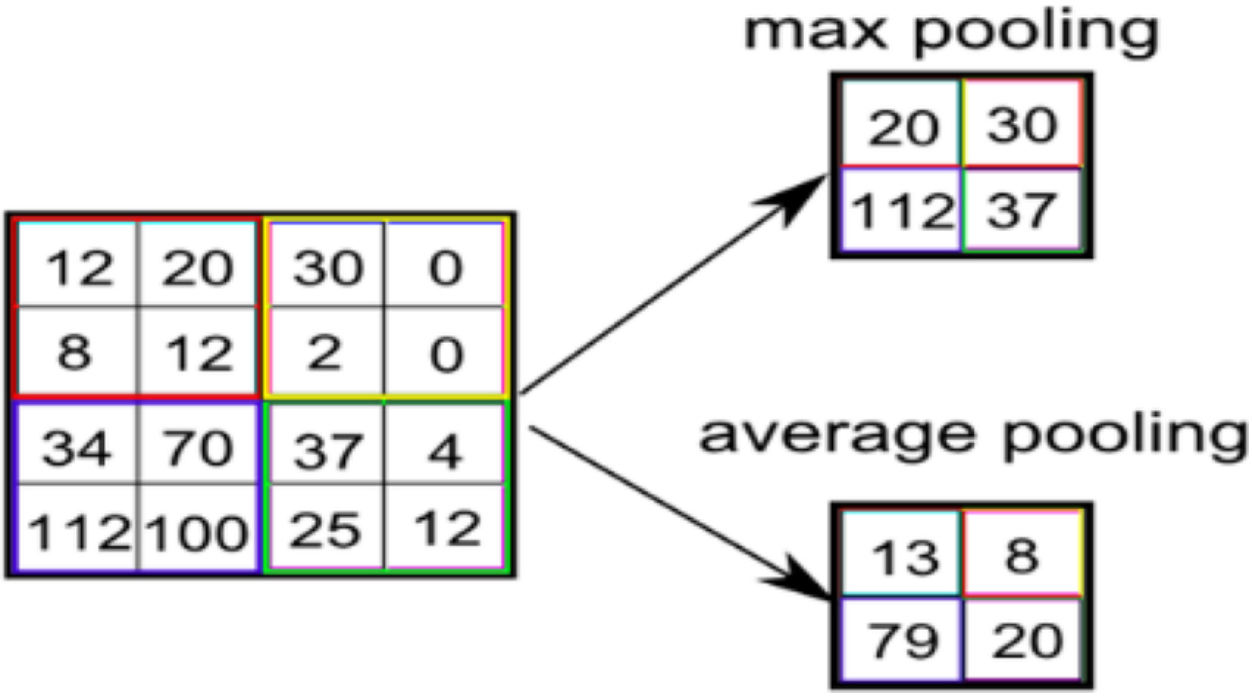


Figure 29: Max and Average Pooling

54 Hierarchical Feature Learning

CNNs learn features hierarchically across layers. Early layers detect simple patterns such as edges and textures. Intermediate layers capture object parts and shapes. Deeper layers encode high-level semantic concepts.

This hierarchical structure enables CNNs to generalize across variations in scale, orientation, and illumination.

55 Fully Connected Layers

After convolutional and pooling layers, feature maps are flattened and passed to fully connected layers. These layers integrate extracted features to perform high-level reasoning and final prediction tasks such as classification.

For classification problems, the final layer commonly uses:

- Softmax activation for multi-class classification
- Sigmoid activation for binary classification

56 Overfitting and Regularization in CNNs

Despite parameter sharing, CNNs can overfit when trained on limited data. Overfitting occurs when the model learns noise and spurious patterns from the training data rather than generalizable features.

Common regularization techniques include data augmentation, dropout, and L2 weight decay. Pooling layers themselves act as a form of regularization by reducing model sensitivity to small spatial variations.

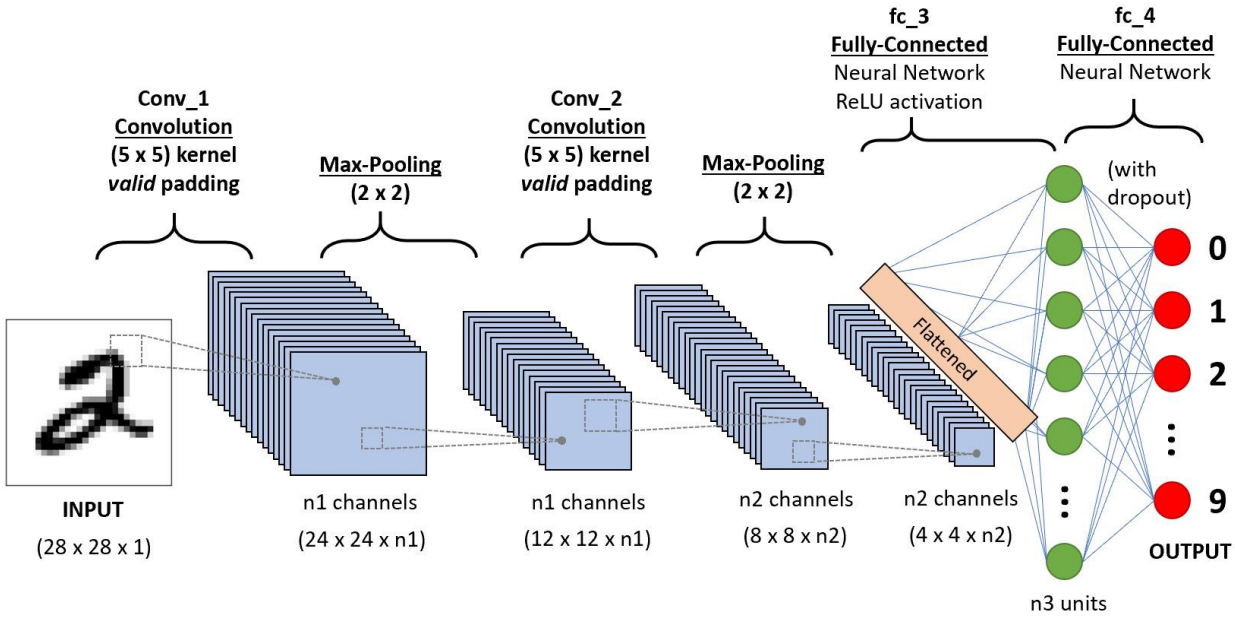


Figure 30: Illustration of a convolutional layer operation

57 Introduction to PyTorch Modules and Automatic Differentiation

PyTorch is a widely used open-source deep learning framework that provides flexible and efficient tools for building, training, and deploying machine learning models. It is particularly popular in research and academia due to its intuitive design, dynamic computation graph, and seamless integration with Python.

This section explains the fundamental PyTorch modules with a focus on tensors, automatic differentiation, and gradient handling. Additionally, it discusses why PyTorch is often preferred over TensorFlow in modern deep learning workflows.

58 The `torch` Module

The `torch` module forms the backbone of the PyTorch framework. It provides support for numerical computation using multi-dimensional arrays known as tensors. Along with tensor operations, the module includes mathematical functions, random number generators, linear algebra utilities, and GPU acceleration support.

Unlike traditional numerical libraries, PyTorch is designed specifically for machine learning and supports automatic differentiation, which is essential for training neural networks.

59 Tensors in PyTorch

59.1 What is a Tensor?

A tensor is a multi-dimensional data structure that generalizes scalars (0D), vectors (1D), matrices (2D), and higher-dimensional arrays. In PyTorch, tensors are the fundamental data units used to represent inputs, parameters, and outputs of neural networks.

$$\text{Scalar} \rightarrow \text{Vector} \rightarrow \text{Matrix} \rightarrow \text{Tensor}$$

59.2 Why Tensors are Needed

Tensors allow efficient representation and manipulation of large-scale numerical data. They support:

- Parallel computation on GPUs
- Batch processing of data
- Automatic gradient computation

These features make tensors ideal for deep learning tasks involving large datasets and complex models.

59.3 PyTorch Tensors vs NumPy Arrays

Although PyTorch tensors are similar to NumPy arrays, they offer two key advantages:

- Native GPU support using CUDA
- Built-in automatic differentiation

This makes PyTorch tensors more suitable for training neural networks.

60 Automatic Differentiation: `autograd`

60.1 Overview of Autograd

PyTorch uses an automatic differentiation engine called `autograd` to compute gradients of tensors during backpropagation. When operations are performed on tensors with gradient tracking enabled, PyTorch constructs a dynamic computational graph.

For a loss function $\mathcal{L}$ dependent on model parameters w, autograd computes:

$$\frac{\partial \mathcal{L}}{\partial w}$$

This computation is handled automatically without explicitly deriving gradient equations.

60.2 Dynamic Computation Graph

PyTorch builds the computational graph dynamically during runtime. This allows:

- Easy debugging
- Conditional execution
- Variable-length input handling

This design is one of PyTorch's strongest advantages over static graph frameworks.

61 Gradient Storage using `.grad`

When the `backward()` function is called on a scalar loss tensor, PyTorch computes gradients for all tensors involved in the computation that have `requires_grad=True`.

The computed gradient is stored in the `.grad` attribute:

$$\texttt{tensor.grad} = \frac{\partial \mathcal{L}}{\partial \texttt{tensor}}$$

These gradients are used by optimization algorithms to update model parameters. PyTorch accumulates gradients by default, so they must be reset before each training iteration.

62 Disabling Gradient Tracking: `torch.no_grad()`

During model evaluation or inference, gradient computation is unnecessary. PyTorch provides the `torch.no_grad()` context manager to disable gradient tracking.

62.1 Why `no_grad()` is Important

Using `torch.no_grad()`:

- Reduces memory consumption
- Speeds up computation
- Prevents unnecessary graph construction

This mode is commonly used during validation and testing phases of a machine learning pipeline.

63 Training vs Inference in PyTorch

- **Training Phase:** Gradients enabled, backpropagation performed
- **Inference Phase:** Gradients disabled, forward pass only

This clear separation improves performance and ensures correct model behavior.

64 Why PyTorch is Preferred over TensorFlow

64.1 Ease of Use

PyTorch follows a Pythonic design philosophy, making it intuitive and easy to learn. The code structure closely resembles standard Python programming, improving readability and debugging.

64.2 Dynamic Graph Execution

Unlike TensorFlow's traditional static computation graph, PyTorch uses dynamic graphs, enabling flexible model design and easier experimentation.

64.3 Research and Community Support

PyTorch is widely adopted in academic research and is the framework of choice for many state-of-the-art models. It has strong community support and frequent updates.

64.4 Debugging and Development Speed

PyTorch allows step-by-step execution and debugging using standard Python tools, significantly reducing development time.

65 Conclusion

This comprehensive report has covered the fundamental concepts of computer vision and deep learning, from basic image processing techniques to advanced neural network architectures. We have explored:

- Mathematical foundations of digital images and their representations
- Spatial and frequency domain analysis techniques
- Feature extraction methods including color, shape, and texture descriptors
- Classical computer vision algorithms for edge detection, corner detection, and shape recognition
- Supervised and unsupervised machine learning concepts
- Neural network fundamentals including perceptrons, activation functions, and backpropagation
- Convolutional Neural Networks for image analysis
- Practical implementation aspects using PyTorch

The field of computer vision and deep learning continues to evolve rapidly, with new architectures and techniques emerging regularly. However, the foundational concepts presented in this report remain essential for understanding and applying these technologies effectively. Whether working on image classification, object detection, or any other vision task, a solid grasp of these principles is crucial for success in this exciting field.